Asynchronous Wait-Free Runtime Verification and Enforcement of Linearizability

ARMANDO CASTAÑEDA, Instituto de Matemáticas, Universdad Nacional Autónoma de México, Mexico
GILDE VALERIA RODRÍGUEZ, Posgrado en Ciencia e Ingeniería de la Computación, Universidad Nacional Autónoma de México, Mexico

arXiv:2301.02638v3 [cs.DC] 13 Nov 2025

This paper presents a theoretical study of the problem of verifying linearizability at runtime, where one seeks for a concurrent algorithm for verifying that the current execution of a given concurrent shared object implementation is linearizable. It shows that it is impossible to runtime verify linearizability for some common sequential objects, regardless of the consensus power of base objects. Then, it argues that a variant of the problem, which we call predictive verification, can be solved, if linearizability is verified indirectly. Namely, it shows that (1) linearizability of a class of concurrent implementations can be predictively verified using only read/write base objects (i.e. without the need of consensus), and (2) any implementation can be transformed to its counterpart in the class using only read/write objects. As far as we know, this is the first runtime verification algorithm for any correctness condition that is fully asynchronous and fault-tolerant. As a by-product, it is obtained a simple and generic methodology for deriving linearizable implementations that runtime verify their responses, and are able to produce a history certifying this, properties that allows the design of concurrent systems in a modular manner with accountable and forensic guarantees. We call such implementations self-enforced linearizable. The results hold not only for linearizability but for a correctness condition that includes generalizations of it such as set-linearizability and interval-linearizability.

CCS Concepts: • **Theory of computation** → **Concurrency**; **Computability**; **Concurrent algorithms**; **Algorithm design techniques**; • **Computing methodologies** → **Concurrent algorithms**; **Distributed algorithms**.

Additional Key Words and Phrases: Concurrent algorithms, Distributed runtime verification, Enforcement, Fault-tolerance, Linearizability, Lock-freedom, Monitoring, Shared memory, Verification, Wait-freedom

ACM Reference Format:
Armando Castañeda and Gilde Valeria Rodríguez. 2025. Asynchronous Wait-Free Runtime Verification and Enforcement of Linearizability. 1, 1 (November 2025), 36 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

1.1 Linearizability and its challenges

Linearizability [62] is the de facto correctness condition for asynchronous shared memory concurrent implementations of objects defined through sequential specifications. Intuitively, an implementation is linearizable if each operation happens atomically at a single moment of time between its invocation and response. Designing linearizable implementations is a simple task due to Herlihy's Universal Construction [59], although the resulting solutions typically do not scale well in practice.

⋆ The conference version of this paper can be found in [19].

Authors' addresses: Armando Castañeda, Instituto de Matemáticas, Universdad Nacional Autónoma de México, Mexico City, Mexico, armando.castaneda@im.unam.mx; Gilde Valeria Rodríguez, Posgrado en Ciencia e Ingeniería de la Computación, Universidad Nacional Autónoma de México, Mexico City, Mexico, gilde@ciencias.unam.mx.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Association for Computing Machinery.
XXXX-XXXX/2025/11-ART $15.00
https://doi.org/XXXXXXX.XXXXXXX

In contrast, designing linearizable *and* scalable implementations is a challenging task, as it requires the use of fine-grained locking or the absence of locks at all, in order to exploit the parallelism in concurrent systems, which typically derives in a large number of scenarios and subtle corner cases that need to be considered in correctness proofs [61, 78, 85].

The importance of linearizability, and more broadly of correct concurrent software, naturally calls for formal verification techniques. Over the past years, a variety of techniques for verifying linearizability have been developed, using different approaches and providing different levels of guarantees. See for example survey [27]. Despite of all efforts, verifying correctness of linearizable implementations remains difficult. Model checking is feasible only for small cases (i.e. for a bounded number of processes, and/or invocations to operations), and finding linearization points, simulations and invariants is hard, sometimes requiring deep understanding of the verified implementation. The problem has been also studied from a theoretical perspective. It is known that deciding whether an implementation is linearizable might be EXPSPACE-complete or even undecidable [14], while deciding if a given finite execution is linearizable might be NP-complete [51, 82].

1.2 Distributed runtime verification and its challenges

Runtime verification is a dynamic, lightweight, yet rigorous, formal method that complements static verification techniques with a more practical approach. It only seeks to verify that the current execution of a system is correct, and maybe prevent an incorrect action or enforce a correct behavior otherwise. The system under inspection can be of any type, from hardware to software, centralized or distributed. We refer the reader to [6, 36, 56, 71] for a detailed exposition of the field.

Broadly speaking, in runtime verification, two, non necessarily disjoint tasks need to be accomplished [13]: (1) the design of a *communication interface* that detects the current execution of the *underlying system* under inspection, and (2) the design of a *monitoring system* that verifies whether the detected execution is correct with respect to some correctness criteria. A big source of difficulty is that the underlying system and the communication interface are typically *decoupled*. Namely, the underlying system is designed, implemented and deployed without considering that in the future a runtime verification mechanism might be integrated to it, hence it might not export enough data of the current execution from which a communication interface and a monitoring system can be built later on. The situation gets more difficult when the underlying system is *distributed* as no process of the system "knows" what the current execution is, each process has only a *partial view* of what the execution could be. The problem is even worse if we seek for a *distributed, asynchronous and fault-tolerant* communication interface, as the processes might not even have the ability to agree on the partial view of a process of the underlying system (e.g. [37]). In such scenario, there are several computational entities that exchange information, subject to delays and failures, yet they have to make consistent decisions. Namely, we have a distributed computing problem.

Designing distributed runtime verification algorithms that are asynchronous and fault-tolerant is a challenging problem (see [13, 28, 33, 44, 72, 90]). In fact, there are only runtime verification solutions (for a number of properties) that are *failure-free* and *synchronous* (e.g. [4, 91]), fault-tolerant with *timing assumptions* (e.g. [7, 9, 10, 12, 29, 34, 42, 43, 49, 65, 88]), or asynchronous *failure-free* (e.g. [21, 38, 40, 73, 93, 96]). That is, the known distributed runtime verification algorithms are *not fully asynchronous and fault-tolerant*. As far as we know, runtime verification of linearizability has only been studied in [30, 31], in asynchronous failure-free models with centralized monitoring systems.

1.3 Asynchronous wait-free runtime verification of linearizability

We are interested in runtime verification of linearizability, where the underlying system $\mathcal{A}$ is an asynchronous concurrent[1] implementation of an object, and the communication interface and monitoring system are consolidated in an asynchronous wait-free shared memory concurrent algorithm $\mathcal{V}$. *Wait-free* [59] means that the algorithm tolerates any number of crash failures. Moreover, we focus on the case where the underlying system $\mathcal{A}$ is a *black-box*, i.e., we do not have access to its specification/pseudocode, hence the only way to obtain information of the current execution is by analyzing the sequence of invocations and responses each process obtains from it.

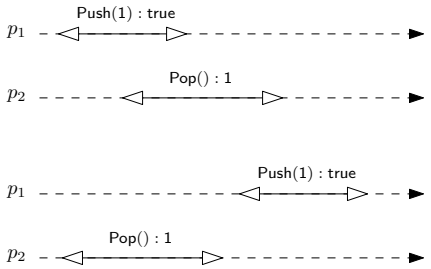


Fig. 1. Two executions of a stack where the two processes have the same partial views, i.e. sequence of invocations and responses; while the execution at the top is linearizable, the execution at the bottom is not. Real-time, unaccessible to the processes, is what ultimately defines the executions.

Why is this setting interesting? While arguably $\mathcal{A}$ and $\mathcal{V}$ being asynchronous and $\mathcal{A}$ being a black-box model the challenging scenario described above, where the system under inspection and the communication interface are decoupled, $\mathcal{V}$ being asynchronous and wait-free guarantees that the progress and timing properties of $\mathcal{A}$ are preserved. For example, if $\mathcal{A}$ is non-blocking (e.g., wait-free) but $\mathcal{V}$ is blocking (maybe because it lock-based), the system that results of integrating $\mathcal{A}$ and $\mathcal{V}$ will be blocking, i.e. non fault-tolerant, hence weakening $\mathcal{A}$'s progress property. Similarly, $\mathcal{A}$ might be designed as an asynchronous implementation for sake efficiency, hence if $\mathcal{V}$ is synchronous or semi-synchronous, $\mathcal{A}$'s timing property is weakened. Indeed, a target in runtime verification is to design communication interfaces and monitoring systems that are "as less intrusive as possible"; ideally they should not interfere with the behavior of the underlying system (e.g. [6, 11, 26, 98]).

In presence of asynchrony and failures, is it possible to capture the actual execution of $\mathcal{A}$ in order to verify linearizability? The answer is clearly no. After the seminal work of Lamport [67], we know that it is *impossible* to determine the order of *non-causally related* events in fully asynchronous distributed systems, and invocations to and responses from $\mathcal{A}$ are local events, which are non-causally related. The partial view of a process in $\mathcal{A}$'s actual execution is just the sequence with its invocation and responses, and for the processes is just impossible to access the real-time order of these local events, which ultimately defines $\mathcal{A}$'s execution (see Figure 1). This reasoning seems to imply that very little should be possible when considering linearizability, as in the end this correctness condition totally depends on real-time order of locals events. However, there are ways to runtime verify linearizability, as we show here.

[1]Through the paper, we interchangeably use the words 'distributed' and 'concurrent' to denote situations where several processes communicate each other in order to collectively solve a problem. In the context of shared memory algorithms, we always use 'concurrent'.

1.4 Results

We propose an interactive model for the theoretical study of the problem of distributed runtime verification of correctness conditions. In the model, a concurrent implementation $\mathcal{A}$ interacts with a *verifier* $\mathcal{V}$, which is required to invoke operations of $\mathcal{A}$, receive the corresponding responses, and somehow compute relevant information of the current execution of $\mathcal{A}$, in order to decide whether it is correct or not; the interaction between $\mathcal{A}$ and $\mathcal{V}$ is infinite. The verifier $\mathcal{V}$ *runtime verifies* linearizability, if $\mathcal{V}$ is *sound*, i.e., if $\mathcal{A}$'s current execution is linearizable, $\mathcal{V}$ does not report ERROR, and *complete*, namely, if $\mathcal{A}$'s current execution is not linearizable, $\mathcal{V}$ reports ERROR.

Using a simple indistinguishability argument, we show that it is impossible to runtime verify linearizability for some common objects such as queues, stacks, sets, priority queues, counters and even the consensus problem, regardless of the *consensus number* [59] of the base objects used in $\mathcal{V}$. Thus, differently from the case of linearizability for which consensus suffices for obtaining linearizable wait-free implementations for any object [59], wait-free runtime verification of linearizability of some objects is impossible even if consensus is available.

Then, perhaps somewhat surprisingly, we show that a variant of the problem can be solved, and without the need of consensus, if linearizability is verified *indirectly*. Namely, we identify a class of concurrent implementations, called *Distributed Runtime Verifiable* (DRV), such that a variant of the distributed runtime verification problem can be solved with respect to DRV. In the variant, a verifier $\mathcal{V}$ is required to be complete and *predictively sound*, which intuitively means that $\mathcal{V}$ is allowed to report ERROR when $\mathcal{A}$'s current execution is linearizable (i.e., a *false negative*), as long as it "predicts" that a $\mathcal{A}$ is *not linearizable*, namely, $\mathcal{V}$ possesses an execution of $\mathcal{A}$ that is not linearizable (hence that non-linearizable execution is necessarily different from the current one).

Intuitively, any implementation in the class DRV produces a "sketch" of its current execution, additionally to operation outputs. It turns out that such sketches are good enough to predictively verify linearizability, for the class DRV, and using only read/write base objects, which have consensus number one [59] (hence incapable of solving consensus among two or more processes). Linearizability can then be indirectly predictively verified because *any* concurrent implementation $\mathcal{A}$ can be transformed into an implementation $\mathcal{A}^*$ in DRV, using only read/write base objects, such that $\mathcal{A}$ and $\mathcal{A}^*$ have the same progress properties, and $\mathcal{A}$ is linearizable if and only if $\mathcal{A}^*$ is linearizable, both with respect to the same sequential object. Therefore, although verifying linearizability of $\mathcal{A}$ is impossible, it is possible to derive an implementation $\mathcal{A}^*$ from $\mathcal{A}$ that implements the same object and linearizability of $\mathcal{A}^*$ is predictively verifiable.

As a by-product, we obtain a methodology to derive implementations that verify linearizability of any of its response at runtime, or report ERROR if they cannot produce linearizable responses any longer. Moreover, the implementations are able to provide an execution that *certifies* whether the responses so far are linearizable or not. We call such implementations *self-enforced linearizable*. The methodology takes *any* implementation $\mathcal{A}$ and produces an implementation $\mathcal{B}$ using $\mathcal{A}$ as a black-box (to produce operation responses) and read/write objects (to runtime verify $\mathcal{A}$'s responses) such that $\mathcal{A}$ and $\mathcal{B}$ have the same progress properties and $\mathcal{A}$ is linearizable if and only if $\mathcal{B}$ is self-enforced linearizable. Thus, in a concurrent system, one can use the derived self-enforced implementation $\mathcal{B}$ instead of $\mathcal{A}$ with the guarantee that all non-ERROR responses of $\mathcal{B}$ are linearizable, as all of then are runtime verified by $\mathcal{B}$ itself. We are not aware of prior concurrent implementations in the literature with such properties. Due to their capability to certify linearizability of their responses, we believe self-enforced linearizable implementations allow the design of concurrent systems in a modular manner (due to modularity of linearizability [62, 95]) with accountable and forensic guarantees. Remarkably, the proposed methodology is simple and generic, hence prone to being

program synthesized (i.e., automatically constructing $\mathcal{B}$ from $\mathcal{A}$ with $\mathcal{B}$ guaranteed to have the aforementioned properties), a property of interests in runtime verification.

All results hold for a correctness condition GenLin defined here, which includes linearizability, as well as set-linearizability [81] and interval-linearizability [17], both generalizations of linearizability for concurrent objects with no sequential specifications [18].

1.5 Structure of the paper

Once Section 2 states the model of computation, the interactive model for distributed runtime verification and the problem of distributed runtime verification are introduced in Section 3. Then, Section 4 recalls the definition of linearizability and Section 5 shows the impossibility for runtime verification of linearizability for some common objects. A high-level perspective of how the impossibility result is evaded and the definition of the predictive version of the problem appear in Section 6. Sections 7 and 8 implement the high-level strategy described in Section 6. Section 9 presents some extension of our results. Section 10 discusses related work, explaining differences with our results, and Section 11 concludes the paper with a final discussion.

2 MODEL OF COMPUTATION

We consider a standard concurrent shared memory system (e.g. [61, 62]) with $n \geq 2$ *asynchronous* processes, $p_1, \ldots, p_n$, which may *crash* at any time during an execution. All but one process can crash in any execution. The *index* of process p_i is i. Processes communicate with each other by invoking *atomic* operations of shared *base objects* that reside in the shared memory: either simple Read/Write operations, or more complex and powerful Read-Modify-Write operations, such as Fetch&Inc or Compare&Swap. In algorithm descriptions, shared base objects are denoted with uppercase letters; local variables used by a process for performing its local computations are denoted with lowercase letter subscripted with the index of the process.

We do not provide detailed specifications of Read-Modify-Write base operations because (1) the proof of the impossibility result in Theorem 5.1 does not need them, and (2) our algorithms use only standard Read/Write operations. We refer the reader to standard textbooks such as [61] for definitions of those operations. The point of mentioning Read-Modify-Write operations in this section is that Theorem 5.1 holds even if any of those operations are used in a verifier (the definition of verifier appears in Section 3).

For ease of exposition, it is assumed that base objects are of unbounded size. Section 11 explains how this unrealistic assumption can be removed from the proposed algorithms. We consider the possibility that processes have *perfectly synchronized local clocks*. Each process can read its local clock in a local computation step, information that then can be written in the shared memory. We stress that the time that elapses between reading a local clock and writing it in the shared memory is unpredictable, as the system is asynchronous. This assumption is not relevant in our algorithms, however it makes our impossibility result in Theorem 5.1 stronger.

An *implementation* of a concurrent object O (e.g. a queue or a stack), specified in some way (more details in Section 4), is a distributed algorithm $\mathcal{A}$ consisting of n local state machines $A_1, \ldots, A_n$, some of them possibly non-deterministic. Local machine A_i specifies which operations on base objects and local computations p_i executes in order to return a response when it invokes a high-level operation of O (sometimes simply called operation). Each of these base objects operations and local computations is a *step*. Invocations and responses are local computations as well but we do not refer to them as steps.

An *execution* of $\mathcal{A}$ is a possibly infinite sequence of steps, plus invocations and responses to high-level operations of the concurrent object O, with the following *well-formedness* properties:

(1) Each process is sequential, namely, it first invokes a high-level operation, and only when it has a corresponding response, it can invoke another high-level operation.
(2) A process takes steps only between an invocation and a response.
(3) For any invocation of an operation op_i by a process p_i, denoted $inv_i(op_i)$, the steps of p_i between that invocation and its corresponding response (if there is one), denoted $res_i(op_i)$, are steps specified by A_i when p_i invokes op_i.

It is assumed that after a process completes an operation, it non-deterministically picks the operation it executes next. This assumption is to guarantee that correctness is proven considering all possible combinations of invocations to high-level operations of an implementation.

For simplicity, and without loss of generality, we assume that every concurrent object provides a single high-level operation, called Apply, that receives as input *op*, a description of the actual operation that is invoked, which includes the inputs to the actual operation. We also assume, again without loss of generality, that Apply is invoked with a given input *op* only once (a fictitious input value to the actual operation can be added such that all inputs are different).

Typically, invocation and responses of an implementation $\mathcal{A}$ also include $\mathcal{A}$ as part of their description. We however drop that information from invocations and responses because it facilitates our discussion, although some ambiguity is introduced.

An implementation $\mathcal{A}$ can use other implementation $\mathcal{B}$ in order to produce responses, namely, the processes can invoke high-level operation of $\mathcal{B}$. Hence, when a process invokes an operation of $\mathcal{B}$, it continues executing the steps specified by $\mathcal{A}$ only after it receives the corresponding response from $\mathcal{B}$. Consider any execution E of $\mathcal{A}$. We let $E|\mathcal{B}$ denote the sequence of steps, invocations and responses of $\mathcal{B}$ in E. Then, $E|\mathcal{B}$ is a well-formed execution of $\mathcal{B}$. In what follows, unless stated otherwise, when we talk about operations in E, we mean only the operations of $\mathcal{A}$, excluding the nested calls to operations of $\mathcal{B}$.

A high-level operation in an execution is *complete* if both its invocation and response appear in the execution. An operation is *pending* if only its invocation appears in the execution. A process is *correct* in an infinite execution of an implementation if it takes infinitely many steps. When considering infinite executions, we focus on those that are *fair*: for every correct process and every $K \geq 1$, there is a finite prefix with K steps of that process. An implementation $\mathcal{A}$ is *wait-free* if in every infinite execution, every correct process completes infinitely many operations [59]. An implementation $\mathcal{A}$ is *lock-free* if in every infinite execution, infinitely many operations are complete [62]. Thus, a wait-free implementation is lock-free but not necessarily vice versa. We consider only implementations that are at least lock-free. The notions of wait-freedom and lock-freedom naturally extend to specific operations or fragments of pseudocode.

The *step complexity* of an implementation is the maximum number of base operations a process needs to take to produce a response.

Sometimes it will be convenient to think an implementation $\mathcal{A}$ as a *black-box* whose specification cannot be accessed, and hence the only information that can be obtained from it are the executions it produces without steps. We call such executions without steps *histories*, namely, sequences of invocations and responses satisfying the first two well-formedness properties stated above. As in [62], we define an *abstract implementation* as a set of well-formed histories. By abuse of notation, for any execution E of an implementation $\mathcal{A}$, we let E itself denote the history obtained from E (i.e. the sequence obtained by removing from E all its steps), and let $\mathcal{A}$ denote itself the abstract implementation with all histories of $\mathcal{A}$ (i.e., the set with all histories obtained from the executions of $\mathcal{A}$). This abuse of notation will facilitate the discussion, at the cost of introducing some ambiguity.

The *consensus number* [59] of a shared object O is the maximum number of processes that can solve the *consensus* problem, using any number of instances of O in addition to any number

of Read/Write base objects. Consensus numbers induce the *consensus hierarchy* where objects are classified according their consensus numbers. The simple Read/Write operations stand at the bottom of the hierarchy, with consensus number one and the lowest coordination power. At the top of the hierarchy we find operations with infinite consensus number, like Compare&Swap, that provide the maximum possible coordination power.

3 AN INTERACTIVE MODEL FOR DISTRIBUTED RUNTIME VERIFICATION

Let us fix a concurrent object O, specified in some way. Let $\mathcal{A}$ be a lock-free implementation of O. Intuitively, a *correctness condition* is a mechanism to separate the correct implementations of O from the incorrect ones. Basically, it is a predicate P_O that *all* finite executions of $\mathcal{A}$ need to satisfy for $\mathcal{A}$ being declared correct, with respect to P_O. It is known that if P_O is linealizability, deciding whether $\mathcal{A}$ is linearizable might be EXPSPACE-complete or even undecidable [14], depending on the object O. Deciding if a given finite history is linearizable is decidable, but it might be NP-complete [51, 82], although for some objects this question can be decided in polynomial time [15, 32]. From now on, we will assume that each process can locally test if a given finite history satisfies P_O.

```
Shared Variables:
  Shared memory M

Operation Verify(A) is
(01) set_i ← ∅
(02) while true do
(03)     op_i ← non-deterministically chosen high-level operation that is not in set_i
(04)     set_i ← set_i ∪ {op_i}
(05)     Encode in M the invocation to Apply(op_i) of A
(06)     Invoke operation Apply(op_i) of A %% Local even of process p_i
(07)     resp_i ← response from operation Apply(op_i) of A %% Local even of process p_i
(08)     Encode the response resp_i in M
(09)     exec_i ← description of the current execution of A in M
(10)     if ¬ P_O(exec_i) then
(11)         report (ERROR, exec_i)
(12)     end if
(13) end while
end Verify
```

Fig. 2. Generic structure of a verifier $\mathcal{V}_O$ for correctness condition P_O (code of process p_i).

Let us suppose the existence of a *client* C (i.e. a concurrent algorithm) that solves some distributed problem using $\mathcal{A}$, assuming that $\mathcal{A}$ is correct with respect to P_O. We would like to design an *intermediate layer* $\mathcal{V}_O$ between C and $\mathcal{A}$ that, from time to time, verifies that the *current* execution of $\mathcal{A}$ is correct, namely, it satisfies the predicate P_O, and reports ERROR otherwise (basically $\mathcal{V}_O$ consolidates a communication interface and a monitoring system in a single entity). If $\mathcal{A}$ is indeed correct, then we would like the client C not to be able to distinguish whether it is interacting with $\mathcal{A}$ or $\mathcal{V}_O$, and hence we require $\mathcal{V}_O$ to be asynchronous and wait-free so that the properties of $\mathcal{A}$ are preserved (e.g., its progress properties).

We model this layer as a concurrent algorithm $\mathcal{V}_O$, called *verifier*, that interacts with $\mathcal{A}$. The generic structure of the interaction appears in Figure 2, where $\mathcal{A}$ is a *black-box*, and hence the only way $\mathcal{V}_O$ can obtain information of $\mathcal{A}$ is by invoking high-level operations of it; namely, the processes *only* invoke operations of $\mathcal{A}$ and *only* receive responses. Thus, $\mathcal{V}_O$ interacts with an *abstract implementation* $\mathcal{A}$ (as defined in Section 2), from which $\mathcal{V}_O$ receives one of its histories in every execution.

During the interaction, each process invokes a series of non-deterministically chosen high-level operations of $\mathcal{A}$ in order to consider all possible combination of operations of $\mathcal{A}$ that any client C

might invoke. For simplicity, every process tests if the history so far satisfies P_O after each of its high-level operations of $\mathcal{A}$ (Line 10). If the predicate is not satisfied, the process reports ERROR together with a *witness* to $\mathcal{A}$, i.e. a history of $\mathcal{A}$ that does not satisfy P_O; in any case, the interaction continues. (In a practical setting the interaction would stop, and reporting ERROR and the witness would be returned to the client C; for simplicity, in our model the interaction continues.) Naturally, the processes in $\mathcal{V}_O$ need to exchange information in order to obtain a description of the current history of $\mathcal{A}$. Thus, each process might store information in the shared memory before and after each of its invocations to and responses from $\mathcal{A}$ (Lines 05 and 08).[2]

Since $\mathcal{A}$ is a black-box in the generic verifier in Figure 2, $\mathcal{V}_O$ cannot be designed for a specific implementation $\mathcal{A}$, it must work for *any* possibly abstract concurrent implementation (even if it is correct with respect to an object $O' \neq O$). This requirement is modeled as *conceptually* $\mathcal{V}_O$ taking $\mathcal{A}$ as its input to the computation.

We restrict our attention to verifiers where the block codes in Lines 03–05 and Lines 08–12 are wait-free. We say that such verifiers are *wait-free*. The *step complexity* of a wait-free verifier is the maximum number of base operations a process needs to take in order to complete one iteration of the while loop, discarding steps in $\mathcal{A}$.

In the definition of the distributed runtime verification problem below, for simplicity, it is assumed that *no process crashes*, and hence all executions of a verifier are infinite, since we are assuming that $\mathcal{A}$ is lock-free or wait-free. The assumption makes the problem easier to state. The possible crashes that can occur in the system are modeled by the fact (1) we focus on wait-free verifiers, (2) the system is asynchronous and (3) correctness is tested at finite prefixes of a given infinite execution. The soundness and completeness properties that specify the problem have been considered in the past (see for example [28, 79, 80]); the definition basically adapts them to fit in our interactive setting.

Definition 3.1 (Distributed Runtime Verification). *Let O be a concurrent object specified in some way, and consider a correctness condition P_O for O. We say that a* wait-free *verifier $\mathcal{V}_O$* distributed runtime verifies *P_O if the following two requirements are fulfilled in every infinite execution E of $\mathcal{V}_O$ with an arbitrary input (abstract) implementation $\mathcal{A}$:*

(1) *Soundness: If for every finite prefix E' of E, $E'|\mathcal{A}$ satisfies P_O, then no process reports* ERROR.
(2) *Completeness. If E has a finite prefix E' such that $E'|\mathcal{A}$ does not satisfy P_O, then at least one process reports* ERROR *together with a witness to $\mathcal{A}$.*

We say that P_O is distributed runtime verifiable *if there is a wait-free verifier that distributed runtime verifies P_O.*

The previous definition is flexible, it is not difficult to modify it to cover the cases where $\mathcal{A}$ is blocking [61] (i.e. it internally uses locks) or obstruction-free [61] (namely, progress is guaranteed only when a process runs solo), or correctness conditions for one-shot distributed problems such as tasks [60], where each process invokes one high-level operation. The main difference is that in these cases, the interaction might be only finite.

Below, for sake of compactness, sometime we will simply say verify/verifiable instead of distributed runtime verify/verifiable.

4 LINEARIZABILITY

Linearizability [62] is the de facto standard correctness condition for concurrent implementations of objects defined with sequential specifications. It extends the concept of *atomicity* [69, 70] to

[2]The communication in Lines 05 and 08 ultimately creates causal relations [67] between invocations to and responses from $\mathcal{A}$, whose aim is to compute the current execution of $\mathcal{A}$.

any sequential object. Intuitively, an execution of an implementation is linearizable if its operations can be ordered sequentially, without reordering non-overlapping operations, so that their responses satisfy the sequential specification of the implemented object. Figure 3 depicts examples of linearizable and non-linearizable histories of a stack implementation.

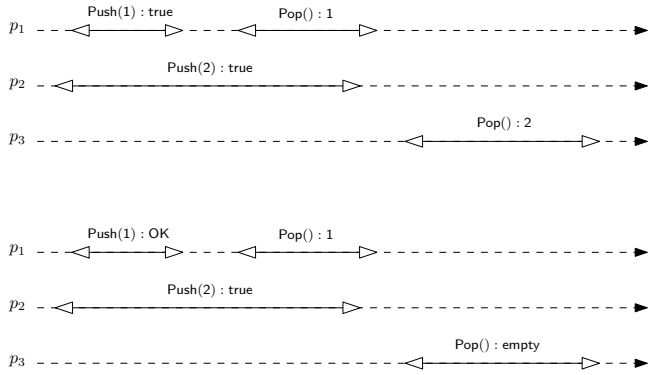


Fig. 3. Two 3-process histories of a stack implementations are depicted. Time goes from left to right, and operations are denoted with a double-ended arrows. For clarity, each operation Apply(op) is simply denoted op. The execution at the top is linearizable with respect to the usual sequential specification of a stack; a linearization of it is: ⟨Push(2) : true⟩⟨Push(1) : true⟩⟨Pop() : 1⟩⟨Pop() : 2⟩. The execution at the bottom is not linearizable because the stack cannot be empty when the operation ⟨Pop() : empty⟩ starts.

Definition 4.1 (Sequential Specifications). *A* sequential specification *of a concurrent object O is a state machine specified through a (possibly partial) transition function δ. Given a state q and an invocation $inv_i(op_i)$ of process p_i, $\delta(q, inv_i(op_i))$ returns the tuple $(q', res_i(op_i))$ indicating that the machine moves to state q' and the response to op_i is $res_i(op_i)$. If the machine is* non-deterministic*, $\delta(q, inv_i(op_i))$ is a set of tuples from which one of them is non-deterministically chosen to respond to $inv_i(op_i)$ and move the machine to the next state. The sequences of invocation-response tuples, $\langle inv_i(op_i) : res_i(op_i)\rangle$, produced by the state machine are its* sequential histories.

For sake of clarity, a tuple $\langle inv_i(op_i) : res_i(op_i)\rangle$ is simply denoted op_i. Also, subscripts of invocations and responses are omitted.

A history E' is an *extension* of a finite history E, if E' can be obtained from E by appending zero or more responses for some of E's pending operations.

For any history E and any process p_i, $E|p_i$ denotes the sequence of invocations and responses of p_i in E. Two histories E and F are *equivalent* if $E|p_i = F|p_i$, for every process p_i.

For any finite history E of an implementation $\mathcal{A}$, $comp(E)$ denotes the history obtained by removing from E all invocations of pending operations; note that $comp(E)$ is well-formed. To formalize linearizability, we define a partial order $<_E$ on the *complete* operations of any history E: $op <_E op'$ if and only if $res(op)$ precedes $inv(op')$ in E. Two complete operations are *concurrent* if they are incomparable by $<_E$. The history is *sequential* if $<_E$ is a total order.

We consider the definition of linearizability in [95], which is a slight variant of the original definition [62] that fixes some corner cases.

Definition 4.2 (Linearizability). *Let O be any concurrent object. A finite history E is* linearizable *with respect to O if there is an extension E' of E and a sequential history S of O such that*

(1) *$comp(E')$ and S are equivalent and*

(2) $<_{comp(E')} \subseteq <_S$.

The sequential history S is said to be a linearization *of E. We say that an implementation $\mathcal{A}$ is* linearizable *with respect to O, if each of its finite histories is linearizable with respect to O.*

5 LINEARIZABILITY IS NOT RUNTIME VERIFIABLE

This section shows that for some common objects such queues, stacks, priority queues, counters, and even the fundamental consensus problem, linearizability is not distributed runtime verifiable, regardless of the consensus number of the base objects the processes use in a verifier. The following simple impossibility proof captures informal arguments that have been used in the past (e.g. [28, 48]) to argue that distributed runtime verification of some correctness conditions in asynchronous distributed systems is impossible.

Theorem 5.1 (Impossibility for Distributed Runtime Verification of Linearizability). *For queues, stacks, sets, priority queues, counters and the consensus problem (defined as sequential objects) linearizability is not distributed runtime verifiable, regardless of the consensus number of base objects used in a verifier.*

Proof. We focus on the case of the queue as all other cases are very similar. By contradiction, suppose that there is a wait-free verifier $\mathcal{V}_{\text{queue}}$ that verifies linearizability for queues. Consider the following *non-linearizable* implementation $\mathcal{A}$: every Enqueue operation returns true, and every Dequeue operation returns empty, for every process $p_i \neq p_2$, and for p_2, it returns 1 in its first operation, and returns empty in every subsequent operation.

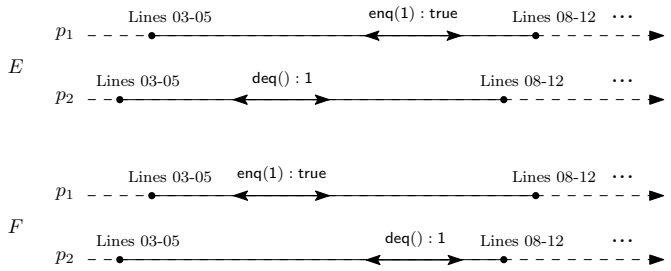


Fig. 4. Executions E and F in the impossibility proof for $n = 2$.

We will exhibit two executions E and F of $\mathcal{V}_{\text{queue}}$ with input $\mathcal{A}$ and argue that $\mathcal{V}_{\text{queue}}$ cannot simultaneously satisfy the soundness and completeness requirements of the distributed runtime verification problem. We use the generic structure in Figure 2 to describe the executions.

Execution E is the next (see Figure 4):

(1) In its first iteration of the while loop, process p_2 picks op_2 = Dequeue() in Line 03, executes the local step in Line 04 and the base operations corresponding to Line 05.
(2) In its first iteration of the while loop, process p_1 picks op_1 = Enqueue(1) in Line 03, executes the local step in Line 04 and the base operations corresponding to Line 05.
(3) Process p_2 executes Lines 06 and 07 of its first iteration of the while loop. Thus it obtains response $resp_2 = 1$ for its high-level operation op_2 = Dequeue().
(4) Process p_1 executes Lines 06 and 07 of its first iterations of the while loop. Thus it obtains response $resp_1$ = true for its high-level operation op_1 = Enqueue(1).
(5) Process p_2 executes Lines 08 to 12 of its first iteration of the while loop.

(6) Process p_1 executes Lines 08 to 12 of its first iteration of the while loop.
(7) For each $k = 1, 2, \ldots, \infty$ (in this order), $p_{(k \mod n)+1}$ executes a whole iteration of the while loop where it picks $op_{(k \mod n)+1} = \text{Dequeue}()$ in Line 03 (and hence the response in Line 07 is $resp_{(k \mod n)+1} = \text{empty}$).

Execution F is similarly constructed, with the exception that the steps 3 and 4 of the previous construction appear in the opposite order (see Figure 4). In other words, in E, the first high-level operations of p_2 is executed first and then the first high-level op_1 is executed, whereas in F, the high-level operations are executed in the opposite order.

The history of $\mathcal{A}$ obtained from every finite prefix of F is linearizable; in contrast, the history of $\mathcal{A}$ obtained from every finite prefix of E containing at least the first operation of p_2, is not.

As the only difference between E and F is the order of occurrence of local events of p_1 and p_2, and this order is not accessible to p_1 and p_2, every process p_i transitions through the same sequence of local states in both executions E and F. This means that the executions are *indistinguishable* to all processes, and hence in both executions they make the same sequence of decisions in Lines 08 to 10. Thus, if no process reports ERROR in E and F, then $\mathcal{V}_{\text{queue}}$ does not fulfills completeness due to E, and if at least one process reports ERROR in E and F, then $\mathcal{V}_{\text{queue}}$ does not fulfills soundness due to F. Therefore, $\mathcal{V}_{\text{queue}}$ cannot exist.

For the other objects, the argument is nearly the same. For example, for the case of the stack, Pop is replaced with Dequeue, and Push is replaced with Enqueue. For the case of the consensus, we define an object with a single Decide operation that can be invoked several times, and the first operations among all processes sets its input as the decision of the consensus. □

The previous proof can easily be extended to variants of linearizability that have been used to specify relaxed versions of sequential objects. Examples of such variants are quasi-linearizabilty [2], k-stuttering [58], set-linearizability [81], interval-linearizability [17] and intermediate value linearizability [86]. The reason is that all these relaxations include the sequential executions with the "exact" sequential behavior of the object that is relaxed, which suffices for the previous proof to hold.

6 EVADING THE IMPOSSIBILITY RESULT: THE STRATEGY AT A HIGH-LEVEL

The rest of the paper is devoted to show that it is possible to circumvent the impossibility in Theorem 5.1. Roughly speaking, it will argue that, for any sequential object O, linearizability of any implementation $\mathcal{A}$ of O can be *indirectly* verified through a class of implementations that we call *Distributed Runtime Verifiable* (DRV). Perhaps somewhat surprisingly, the class DRV can be verified with respect to a *predictive* variant of the distributed runtime verification problem, in which, generally speaking, a verifier $\mathcal{V}$ is allowed to report a *false negative* as long as $\mathcal{V}$ "predicts" that $\mathcal{A}$ is not linearizable, namely, $\mathcal{V}$ has evidence of a non-linearizable execution of $\mathcal{A}$. This section gives a high-level perspective of the strategy followed in the next sections, and introduces the predictive variant of the distributed runtime verification problem.

The impossibility in Theorem 5.1 comes from the inability of the processes in any verifier $\mathcal{V}$ to detect the actual history of an arbitrary implementation $\mathcal{A}$. The history is ultimately defined by the order of occurrence of local events of processes (invocations and responses). Since the processes are asynchronous, a delay of arbitrary length can happen between the steps of a process in Lines 05 and 06, and between the steps in Lines 07 and 08, in the generic verifier in Figure 2. Thus, basically the processes are only able to detect a history of $\mathcal{A}$ where the operations might be "stretched". Figure 5 schematizes this situation when $\mathcal{A}$ is a queue. Essentially, this is the main argument in the proof of Theorem 5.1.

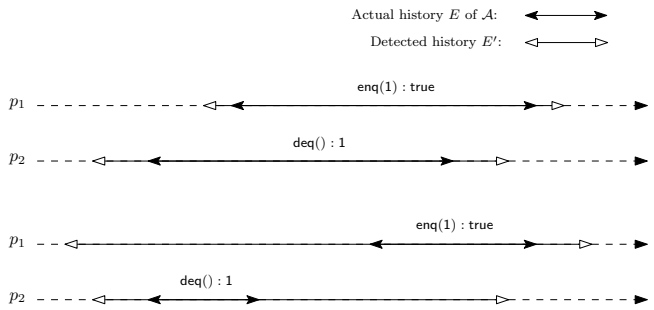


Fig. 5. Due to asynchrony, processes in a verifier might detect histories where operations of $\mathcal{A}$ are "stretched". This phenomenon is exemplified with two examples of a queue implementation where an operation Apply(op) is simply denoted op. At the top, both histories, the actual one E and the detected one E', are linearizable. At the bottom, the actual history E is not linearizable, but the detected history E' is linearizable due to a long delay between the event that announces the operation of $\mathcal{A}$ that p_1 is going to call next (Line 05) and the actual moment when the operation is called (Line 06).

Let E denote the actual history of $\mathcal{A}$ and E' denote the history of $\mathcal{A}$ detected by the processes. Hence, the history E' only "sketches" the actual history E. Actually, it can be that E' is linearizable despite E being non-linearizable, as some operations that do not overlap in E might actually overlap in E' (see the example at the bottom in Figure 5). Thus, we only have the following implication:

$$E \text{ is linearizable} \implies E' \text{ is linearizable.}$$

Arguably, this is the best the processes can do in any verifier $\mathcal{V}$ for detecting the actual history of $\mathcal{A}$. For the time being, let us suppose that the processes somehow can compute E', and are able to write it to and read it from the shared memory. With the help of E', the processes can easily satisfy soundness in a verifier, each process simply needs to locally test whether E' is linearizable. Completeness however cannot be satisfied: if E is not linearizable, E' might or might not be linearizable (see again the example at the bottom in Figure 5). This discussion suggests a weaker version of the distributed runtime verification problem, requiring soundness and a weaker version of completeness where processes are allowed to output *false positives*, i.e., ERROR is not reported although the actual history of $\mathcal{A}$ is not linearizable. This weaker version of the problem can be formally defined and shown to be solvable [64]. However, this result is not completely satisfactory, since, after all, the main motivation of any runtime verification technique is preventing incorrect responses. We can do better, as explained next.

Above we only achieved soundness because we had: E is linearizable $\implies$ E' is linearizable, namely, the implication goes from the actual history to its sketch. If the implication is reversed, we can achieve completeness instead. This is what we do, for an implementation obtained from $\mathcal{A}$. We take the mechanism that computes E' and use it to produce a *new* implementation $\mathcal{A}^*$ that "wraps" $\mathcal{A}$, where processes output the responses obtained from $\mathcal{A}$, somehow together with the detected history E' (each process outputs only a "piece" of E'). It turns out that $\mathcal{A}$ and $\mathcal{A}^*$ have the same progress properties, and $\mathcal{A}$ is linearizable if and only if $\mathcal{A}^*$ is linearizable. We now have that in every history E^* of $\mathcal{A}^*$, E' sketches E^* with the difference that, compared to E^*, the operations in E' might "shrink". Two histories of $\mathcal{A}^*$ when $\mathcal{A}$ is queue are schematized in Figure 6. We thus have:

$$E' \text{ is linearizable} \implies E^* \text{ is linearizable,}$$

as desired.

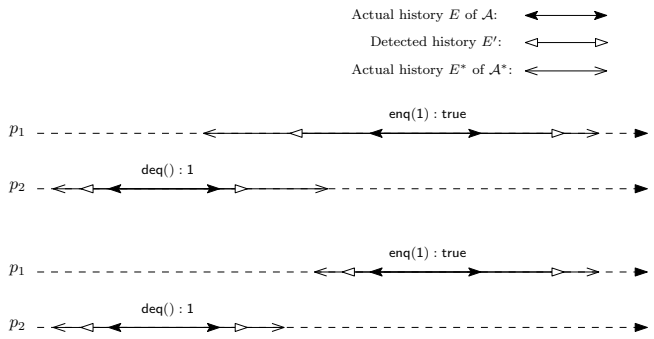


Fig. 6. The operations of any execution E^* of $\mathcal{A}^*$ might "shrink" in the detected history E'. Two executions of $\mathcal{A}^*$ where $\mathcal{A}$ is a queue implementation are shown; an operation Apply(op) is simply denoted op. In the example at the top, the actual history of $\mathcal{A}^*$ is linearizable but the history detected inside $\mathcal{A}^*$ is not. In contrast, in the example at the bottom, the actual history of $\mathcal{A}^*$ is not linearizable, which implies that the detected history is not linearizable either.

Thus, if E^* is not linearizable then its sketch E' (which is part of $\mathcal{A}^*$'s output) is not linearizable either, and hence in a *verifier for the derived implementation* $\mathcal{A}^*$, processes now can fulfill completeness by locally analyzing E' (in the verifier, processes need to exchange information to obtain the whole sketch E'). For soundness, however, it can be the case that E' is not linearizable although E^* is indeed linearizable (see the example at the top in Figure 6). It turns out that E' is a history of $\mathcal{A}^*$ and then it *witnesses* that $\mathcal{A}^*$ is not linearizable. Hence, processes have a reason to report ERROR: the current history of $\mathcal{A}^*$ might be linearizable but the processes somehow have "predicted" that $\mathcal{A}^*$ is not linearizable, and they have a history that witnesses that fact. Moreover, no process can distinguish between E^* and E' (i.e. the histories are equivalent), hence it is actually possible that the actual history of $\mathcal{A}^*$ is E' instead of E^*. Therefore, the processes in a verifier can fulfill a *predictive* version of soundness for $\mathcal{A}^*$, where they are allowed to report ERROR when $\mathcal{A}^*$'s current history is linearizable, as long as this comes with a witness.

The previous discussion motivates the predictive[3] version of the runtime verification problem below, which will be shown to be solvable for implementations such as $\mathcal{A}^*$. DRV will denote the class with the implementations like $\mathcal{A}^*$.

Definition 6.1 (Distributed Runtime Predictive Verification). *Let O be a concurrent object specified in some way, and consider a correctness condition P_O for O. We say that a* wait-free *verifier $\mathcal{V}_O$* distributed runtime predictively verifies *P_O if the following two requirements are fulfilled in every infinite execution E of $\mathcal{V}_O$ with an arbitrary input (abstract) implementation $\mathcal{A}^*$:*

(1) *Predictive Soundness: If for every finite prefix E' of E, $E'|\mathcal{A}^*$ satisfies P_O, then either no process reports* ERROR*, or at least one process reports* ERROR *together with a witness for $\mathcal{A}^*$.*
(2) *Completeness. If E has a finite prefix E' such that $E'|\mathcal{A}^*$ does not satisfy P_O, then at least one process reports* ERROR *together with a witness for $\mathcal{A}^*$.*

We say that P_O is distributed runtime predictively verifiable *if there is a wait-free verifier that distributed runtime predictively verifies P_O.*

[3]We have chosen to use 'predictive' as the modified soundness property is reminiscent to predictive dynamic data-race detection techniques (e.g., [66, 75, 76, 83]), which soundly report data-race violations by analyzing program executions, even if the data-race violation is not observed in an execution.

A slight variant of the proof of Theorem 5.1 shows that the predictive version above is impossible for the general class of implementations (see Appendix A).

An intuition about the construction from $\mathcal{A}$ to $\mathcal{A}^*$. The idea in $\mathcal{A}^*$ is to make asynchrony an "ally" instead of an "enemy". Suppose that in an execution E^* of $\mathcal{A}^*$, the actual history E of $\mathcal{A}$ is not linearizable. If delays are short, then E' is not linearizable (see the example at the bottom of Figure 6), and hence the mistake can be detected. If delays are long, then the sketch E' is linearizable (hence E^* too); in this case the mistake cannot be detected, but E' somehow "enforces" linearizability in E^*. Therefore, a crucial property of $\mathcal{A}^*$ is that the sketches it produces allow any client C that uses $\mathcal{A}^*$ instead of $\mathcal{A}$, detect any possibly non-linearizable history of $\mathcal{A}$ that *is not enforced corect* by $\mathcal{A}^*$. Thus, C's computation can never be compromised by incorrect responses of $\mathcal{A}$, as they are either enforced in $\mathcal{A}^*$ or detectable in C.

7 THE CLASS OF DISTRIBUTED RUNTIME VERIFIABLE (DRV) IMPLEMENTATIONS

This section introduces the class DRV and shows its main properties. Roughly speaking, it will show that (1) every implementation can be easily transformed into its counterpart in DRV and (2) every implementation in DRV provides a "sketch" of the current execution, additionally to the outputs of the object it implements. As we will see, the sketches are the key property that makes any of these implementations predictively verifiable. Furthermore, the same result holds not only for linearizability, but for a class of objects, with its companion correctness condition, that generalize linearizability. After defining the generalized class of objects, the section defines the class DRV and shows the main properties of the implementations in the class.

7.1 Generalizing linearizability

In the rest of the paper, similarly to [62], an *abstract object*, called just *object* for simplicity, is defined as a set of well-formed finite histories. The associated correctness condition is the *membership predicate*. Thus, a finite history of an implementation is *correct* with respect to the object if the history belongs to the set specifying the object. Then, an implementation is *correct* with respect to the object if each of its finite histories belong the set specifying the object.

Linearizability can be alternatively stated through the abstract object formalism:.

Remark 7.1 (Linearizability as abstract objects). *For any sequential object O, consider the abstract object O' with every finite history that is linearizable with respect to O. Then, for any implementation $\mathcal{A}$, it holds that $\mathcal{A}$ is linearizable with respect to O if and only if $\mathcal{A}$ is correct with respect to O' (i.e., every history of $\mathcal{A}$ belongs to O').*

In order to generalize linearizability, we define a partial order on the set of operations, complete *and* pending, of any history E of an implementation; the relation is denoted $\prec_E$. For any two operations op and op' in E, we have that $op \prec_E op'$ if and only if $res(op)$ precedes $inv(op')$ in E. Thus, the only difference with $<_E$ is that $\prec_E$ also relates pending operations.

Definition 7.1 (Similarity between histories). *A finite history E is* similar *to a finite history F if there is a history E' such that:*

(1) *E' can be obtained from E by appending responses to some pending operations and removing invocations of some pending operations,*
(2) *E' and F are equivalent, and*
(3) *$\prec_{E'} \subseteq \prec_F$.*

Definition 7.2 (The class GenLin). *The class* GenLin *contains every abstract object that is closed by prefixes and similarity. Namely, if the set specifying the object contains F, then it also contains:*

(1) *every prefix E of F, and*
(2) *every history E that is similar to F.*

We now show that linearizability is contained in the class GenLin. Namely, for every sequential object O, the abstract object with all finite histories that are linearizable with respect to O, belongs to GenLin.

Lemma 7.1 (Linearizability is contained in GenLin). *Let F be any finite history that is linearizable with respect to some sequential object O (i.e. a state machine). Then,*

(1) *every prefix E of F is linearizable with respect to O, and*
(2) *every history E that is similar to F is linearizable with respect to O.*

Therefore, the abstract object with all finite histories that are linearizable with respect to O is a member of GenLin.

Proof. The prefix closure proof in [53][Theorem 4] assumes a definition of linearizability that is slightly different than the one we use here; that proof however also holds in our case. For completeness, we present the proof. Let $F = EE'$. Consider any linearization S of F. Then, there is an extension F' of F such that $comp(F')$ and S are equivalent and $<_{comp(F')} \subseteq <_S$. Let $S = S_E S_{E'}$, where S_E is the shortest prefix of S with all complete operations in E. We argue that S_E is a linearization of E. Let us observe first that S_E does not have an operation whose invocation appears in E': (1) by definition of S_E, the last operation op of S_E is complete in E, and (2) if S_E has such an operation op', then $op' <_{S_E} op$, which contradicts that $<_{comp(F')} \subseteq <_S$ because we certainly have $op <_{comp(F')} op'$. Now, let E'' be the extension of E obtained by appending all response in S_E to pending operations in E. It is not hard to see that $comp(E'')$ and S_E are equivalent: if there is a p_i such that $comp(E'')|p_i \neq S_E|p_i$, then simply $comp(F')$ and S are not equivalent. We now argue that $<_{comp(E'')} \subseteq <_{S_E}$, from which we conclude that indeed S_E is a linearization of E. Consider any $op <_{comp(E'')} op'$. Since $comp(E'')$ and S_E are equivalent, both op and op' appear in S_E. Moreover, S_E is a sequential history, and hence $<_{S_E}$ must relate op and op'. If both op and op' are complete in E, then $op <_E op'$, hence $op <_F op'$, and consequently $op <_{comp(F')} op'$, and as $<_{comp(F')} \subseteq <_S$, we have $op <_S op'$, which implies $op <_{S_E} op'$. Consider now the case where at least one of op and op' are pending in E. Note that if op is pending in E, then a response to it is appended in E'', and hence it cannot be the case that $op <_{comp(E'')} op'$. Thus, op is complete in E, and op' is pending in E. In F', either a response to op' is appended, or no responses to it is appended because op' is complete in F. In any case, we have $op <_{comp(F')} op'$, and hence $op <_S op'$, as $<_{comp(F')} \subseteq <_S$, which ultimately implies that $op <_{S_E} op'$. Thus, we conclude that $<_{comp(E'')} \subseteq <_{S_E}$, and hence S_E is a linearization of E.

For the second claim, consider any history E' obtained from E by appending responses to some pending operations and removing invocations of some pending operations, with E' and F being equivalent and $\prec_{E'} \subseteq \prec_F$. Let I and R_E denote the sets with the invocations and responses removed and appended, respectively, to obtain E'. Consider any linearization S of F. Then, there is an extension F' of F such that $comp(F')$ and S are equivalent and $<_{comp(F')} \subseteq <_S$. Let R_F denote the set with the responses appended to obtain F'. Let E'' be any extension of E obtained by appending to it the responses in $R_E \cup R_F$. We prove that $comp(E'')$ and S are equivalent and $<_{comp(E'')} \subseteq <_S$, from which follows that S is a linearization of E. As E' and F are equivalent, it is easy to see that $comp(E'')$ and $comp(F')$ are equivalent (just note that the invocations in I do not appear in $comp(F')$), and hence $comp(E'')$ and S are equivalent. Now, consider any $op <_{comp(E'')} op'$. Observe that it cannot be that the response of op is in $R_E \cup R_F$. Then, op is completed in E; however op' might be pending or complete in E. Note that $op <_{comp(E'')} op'$ implies $op \prec_{E'} op'$, and hence

$op \prec_F op'$, as $\prec_{E'} \subseteq \prec_F$, from which follows $op <_{comp(F')} op'$. We thus have $<_{comp(E'')} \subseteq <_{comp(F')}$, and then $<_{comp(E'')} \subseteq <_S$ because $<_{comp(F')} \subseteq <_S$. Therefore, S is a linearization of E. □

Similarly, it can be shown that variants of linearizability such as set-linearizability [81] and interval-linearizability [17], among others, are contained in the class GenLin. The reason is that these variants differ from linearizability only in the properties of the linearization S of a given execution. In some cases, S can "deviate" from sequential executions of state machine O, or it might be the case that S is not necessarily a sequential execution, where several operations can occur simultaneously at the same time, or even an operation can overlap several operations. To adapt the proof above for set-linearizability and interval-linearizability, we only need to replace the sequential execution S with a set-sequential and interval-sequential execution, respectively.

7.2 The class DRV

Let $\mathcal{A}$ be *any* implementation. We already have seen that it is impossible to runtime verify linearizability of $\mathcal{A}$ with respect to some sequential objects. We will now see that $\mathcal{A}$ can be *indirectly* verified through an implementation $\mathcal{A}^*$ that can be easily obtained from $\mathcal{A}$. The implementation $\mathcal{A}^*$ appears in Figure 7, in which processes communicate though a linearizable snapshot object.

Shared Variable:
N = wait-free linearizable snapshot object, with each entry initialized to $\emptyset$

Local Persistent Variable:
set_i = a set initialized to $\emptyset$

Operation Apply(op_i) **is**
(01) $set_i \leftarrow set_i \cup \{(p_i, op_i)\}$
(02) N.Write(set_i)
(03) Invoke operation Apply(op_i) of $\mathcal{A}$
(04) $y_i \leftarrow$ response from operation Apply(op_i) of $\mathcal{A}$
(05) $s_i \leftarrow N$.Snapshot()
(06) $\lambda_i \leftarrow \bigcup_{k \in \{1,2,\ldots,n\}} s_i[k]$
(07) **return** (y_i, λ_i)
end Verify

Fig. 7. From $\mathcal{A}$ to $\mathcal{A}^* \in$ DRV (code of process p_i).

Definition 7.3 (The snapshot object [1]). Snapshot *is a sequential object whose state is a shared array MEM with n entries, one per process, that can be modified through two operations:* Snapshot() *that returns an n-array with the content of all entries of MEM, and* Write(v) *that writes v in $MEM[i]$, where i is the index of the process that invokes the operation.*[4] *The initial state of the snapshot object is* $[\emptyset, \ldots, \emptyset]$.

There are wait-free linearizable snapshot implementations that use only Read/Write base objects, e.g. [1, 63]. Thus, the steps in Lines 02 and 05 can be assumed to be atomic, due to the modular properties of linearizability [62, 95]. In the step complexity analysis in this section and the next one, we consider the implementation in [63], whose step complexity is $O(n)$. There have been proposed snapshot implementations whose theoretical step complexity is worse but with good performance in real-world system (e.g. [99]).

In $\mathcal{A}^*$, every process p_i simply announces in a shared memory the next high-level operation op_i it wants to execute, then obtains a response for op_i using $\mathcal{A}$, atomically reads all operations that have been announced in the snapshot object so far, storing them all together in a (unordered) set λ_i,

[4]Recall that the index of p_i is i.

and finally returns the set together with the response obtained from $\mathcal{A}$. The set λ_i is called the *view* of (p_i, op_i). As we will see, this simple mechanism, the views, succinctly encode a "sketch" of $\mathcal{A}^*$'s current history, and this sketch is what makes $\mathcal{A}^*$ predictively verifiable.

The construction in Figure 7 uses $\mathcal{A}$ as a black-box, hence it is not based on any property of $\mathcal{A}$. Therefore, for *any* implementation $\mathcal{A}$, regardless of the object it implements, one can construct $\mathcal{A}^*$ as described in Figure 7.

Definition 7.4 (The class of Distributed Runtime Verifiable Implementations). DRV *denotes the class of concurrent implementations obtained through the construction in Figure 7.*

7.3 Analyzing $\mathcal{A}^*$

7.3.1 $\mathcal{A}^*$ preserves $\mathcal{A}$'s properties. We first show that $\mathcal{A}^*$ preserves progress and correctness properties of $\mathcal{A}$. In the rest of the section, we disregard the views in the responses of $\mathcal{A}^*$ (i.e. the sets λ_i), unless stated otherwise. Recall that the execution itself of an implementation also denotes the history obtained from it.

Lemma 7.2 (Correctness of $\mathcal{A}^*$). *Consider any implementation $\mathcal{A}$ and any object O in the class* GenLin. *Then, $\mathcal{A}$ is correct with respect to O if and only if $\mathcal{A}^*$ is correct with respect to O (disregarding the views in the responses of $\mathcal{A}^*$). Furthermore, $\mathcal{A}^*$ is lock-free (resp. wait-free) if and only if $\mathcal{A}$ is lock-free (resp. wait-free), and the step complexity of $\mathcal{A}^*$ is the step complexity of $\mathcal{A}$ plus $O(n)$.*

Proof. First, it is easy to see the segment of code in Lines 01 to 02 is wait-free, as well as the segment in Lines 05 to 07. Thus, if $\mathcal{A}$ is lock-free (resp. wait-free) then $\mathcal{A}^*$ lock-free (resp. wait-free), and vice versa. As for step complexity, it executes one Write before invoking $\mathcal{A}$, and one Snapshot after, which can be implemented in $O(n)$ step using the algorithm in [63]. For correctness, we prove each direction separately.

- $\Rightarrow$ Let E be any finite execution of $\mathcal{A}^*$. We show that E is correct, namely, the history obtained from it (denoted E as well) belongs to O. Since $E|\mathcal{A} \in O$ and O is closed by similarity, both by assumption, it suffices to prove that E is similar to history $E|\mathcal{A}$. Consider the history E' obtained from E by: (1) appending to E the response in $E|\mathcal{A}$ to every operations that is pending in E but complete in $E|\mathcal{A}$, and (2) removing every invocation of a pending operation in E that does not appear in $E|\mathcal{A}$. From the definition of E' and the pseudocode in Figure 7, it can be easily verified that E' and $E|\mathcal{A}$ are equivalent. Additionally, it holds that $\prec_{E'} \subseteq \prec_{E|\mathcal{A}}$: if $op \prec_{E'} op'$, then we must have that $res(op)$ precedes $inv(op')$ in $E|\mathcal{A}$ because, in $\mathcal{A}^*$, operation calls to $\mathcal{A}$ are nested in operation calls of $\mathcal{A}^*$, and hence $op \prec_{E|\mathcal{A}} op'$.
- $\Leftarrow$ Let E be any finite history of $\mathcal{A}$. We argue that $E \in O$. Asynchrony in the model guarantees the existence of the following execution E' of $\mathcal{A}^*$:
  (1) for every process p_i, for each of its operations Apply(op_i), the corresponding invocation and the steps from Lines 01 to 02 appear all together right before the invocation to Apply(op_i) of $\mathcal{A}$ in Line 02, and
  (2) for every process p_i, for each of its operations Apply(op_i), the corresponding response and the steps from Lines 05 to 07 appear all together right after the response from Apply(op_i) of $\mathcal{A}$ in Line 04 (if there is one), and
  (3) E and E' are the same history.
  Basically, E' is obtained from E by adding steps of $\mathcal{A}^*$ right before and after the invocations and responses in E. By assumption, $\mathcal{A}^*$ is correct with respect to O, and hence $E' \in O$, from which follows that $E \in O$, as E and E' are the same history.

□

As already explained at the end of Section 6, $\mathcal{A}^*$ can be alternatively understood as a mechanism that "enforces" correctness in *some* incorrect histories of $\mathcal{A}$, as the example in Figure 8 shows. Nevertheless, Lemma 7.2 implies that $\mathcal{A}^*$ cannot enforce correctness in *all* incorrect histories of $\mathcal{A}$. For those histories that it is not able to "fix", the views provide a mechanism to detect they are incorrect, which will be crucial to fulfill the completeness requirement of the distributed runtime predictive verification problem.

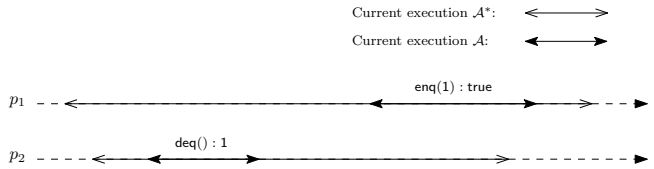


Fig. 8. Due to asynchrony, $\mathcal{A}^*$ is able to "fix" some histories of $\mathcal{A}$. The figure depicts a history of $\mathcal{A}^*$ where the history of $\mathcal{A}$ is not linearizable with respect to the queue, however, the history of $\mathcal{A}^*$ is linearizable. Each operation Apply(op) is simply denoted op and the views of the operations of $\mathcal{A}^*$ are not shown.

7.3.2 *Tight executions.*

Definition 7.5 (Tight executions). *Let $\mathcal{A}^*$ be any implementation in the class* DRV*, and consider any finite execution E of it. We say that E is* tight *if:*

(1) *each of its pending operations has its* Write *step in Line 02 but no* Snapshot *step in Line 05,*
(2) *the invocation and local step in Line 01 of every operation, complete or pending, appear in a sequence right before its* Write *step in Line 02, and*
(3) *the local steps in Lines 06 and 07 and response of every complete operation appear in a sequence right after its* Snapshot *step in Line 05.*

Basically, in a tight execution, the beginning and end of an operation are identified with the Write and Snapshot steps in Lines 02 and 05, respectively.

Any finite execution E of $\mathcal{A}^*$ can be transformed into a tight execution $T(E)$ of $\mathcal{A}^*$ as described next:

(1) remove the invocation and local step in Line 01 of every pending operation with no Write step in Line 02,
(2) for each of the remaining operations, complete or pending, move forward the invocation and local step in Line 01 in a sequence right before its Write step in Line 02,
(3) for each of the remaining operations with its Snapshot step in Line 05, first move backwards the local steps in Lines 06 and 07 and response (if there are any) in a sequence right after its Snapshot step in Line 05, and then insert (if necessary) the missing local steps in Lines 06 and 07 and response to complete the sequence that makes the operation complete.

We say that $T(E)$ is the *tight execution associated* to E.

Observe that indeed $T(E)$ is an execution of $\mathcal{A}^*$: all invocations, responses and steps that are moved, forward of backward, or inserted are *local* to processes, and hence it is immaterial when they occur, or if they occur. Moreover, in E and $T(E)$, the operations obtain the same responses from $\mathcal{A}$ and compute the same views, as the order of invocations to and responses from $\mathcal{A}$ are not modified to obtain $T(E)$, neither the order of Write and Snapshot steps. Intuitively, the difference between E and $T(E)$ is that operations in $T(E)$ span a possibly "shorter" interval of time.

Lemma 7.3 (Tight executions and actual executions). *Let E be any finite execution of $\mathcal{A}^*$. Then, $T(E)$ is an execution of $\mathcal{A}^*$, and for any object O in* GenLin*,*

$$E|\mathcal{A} \in O \implies T(E) \in O \implies E \in O.$$

Proof. As already argued, $T(E)$ is an execution of $\mathcal{A}^*$. We will show that (history) $T(E)$ is similar to (history) $E|\mathcal{A}$, and (history) E is similar to (history) $T(E)$. These two facts will prove the implications because O is closed by similarity, as it belongs to GenLin.

We argue first that E is similar to $T(E)$. Let E' be the history obtained from E by: (1) removing the invocation of every pending operation that is removed from E to obtain $T(E)$, and (2) appending the response in $T(E)$ of every pending operation in E whose response is inserted to obtain $T(E)$. Basically, E' is obtained following steps (a) and (c) in the construction from E to $T(E)$. Note that E and $T(E)$ are equivalent. Consider now any $op \prec_{E'} op'$. Hence $res(op)$ precedes $inv(op')$ in E'. Observe that it cannot be that $res(op)$ is appended to E to obtain E', and $T(E)$, and hence op is complete in E; op' is complete or pending. We must have that $op \prec_{T(E)} op'$ becase to obtain $T(E)$, responses of complete operations might only moved backward and invocations of complete or pending operations might only moved forward. Therefore we have $\prec_E \subseteq \prec_{T(E)}$, from which follows that E is similar to $T(E)$.

We show now that $T(E)$ is similar to $E|\mathcal{A}$. Consider the history E' obtained from $T(E)$ by: (1) removing every invocation of a pending operation in $T(E)$ that does not appear in $E|\mathcal{A}$ (any such operation executes its Write step in Line 02 but does not executes the invocation in Line 03), and (2) appending the response in $E|\mathcal{A}$ to every operations that is pending in $T(E)$ but complete in $E|\mathcal{A}$ (any such operation operation executes its response in Line 04, but does not execute its Snapshot step in Line 05). Observe that E' and $E|\mathcal{A}$ are equivalent. It is also true that $\prec_{E'} \subseteq \prec_{E|\mathcal{A}}$: if $op \prec_{E'} op'$, then we must have that $res(op)$ precedes $inv(op')$ in $E|\mathcal{A}$ because in $\mathcal{A}^*$ operation calls to $\mathcal{A}$ are nested between the Write and Snapshot steps in Lines 02 and 05, from which follows that $op \prec_{E|\mathcal{A}} op'$. Therefore, $T(E)$ is similar to $E|\mathcal{A}$. □

Lemma 7.3 suggests a way to predictively verify $\mathcal{A}^*$, through its tight executions; the idea is actually simple. Suppose that somehow processes are able to compute $T(E)$ of any execution E of $\mathcal{A}$. First, Lemma 7.3 shows that $T(E)$ is a history of $\mathcal{A}^*$. If E is not correct, then the lemma also implies that $T(E)$ is not correct either, and hence it is a witness for $\mathcal{A}^*$, which can be reported to satisfy completeness; and if E is correct, then $T(E)$ might be correct or not, but in either case predictive soundness can be satisfied because if $T(E)$ is not correct, it is a witness for $\mathcal{A}^*$ that can be reported.

In the rest of the section, we argue that the views encode the tight execution associated to the actual execution of $\mathcal{A}^*$, which opens the possibility to implement the simple idea just described.

7.3.3 From views to tight executions and vice versa. Let E be any finite execution of $\mathcal{A}^*$. In the associated tight execution $T(E)$, let us replace each invocation of p_i to operation $\mathsf{Apply}(op_i)$, with the *invocation pair* (p_i, op_i), and replace each response from operation $\mathsf{Apply}(op_i)$ to p_i (if there is one) with the set with all invocation pairs (p_j, op_j) that precedes the response. Figure 9 depicts an example of the replacement. Since invocations and responses in $T(E)$ are associated to the Write and Snapshot steps in Lines 05 and 02, respectively, the view returned by any operation in $T(E)$ is exactly the set just defined. Below, we show that a "sketch" of the history $T(E)$ can be directly obtained from the views of operations in E. First, from the specification of the linearizable Snapshot object and the pseudocode of $\mathcal{A}^*$, we obtain the following:

Remark 7.2 (Properties of views). *Consider the views λ_i and λ_j in the responses of any pair of completed operations $\mathsf{Apply}(op_i)$ and $\mathsf{Apply}(op_j)$ by p_i and p_j (possibly with $p_i = p_j$) in any execution of $\mathcal{A}^*$. The next properties are satisfied.*

(1) *Self-inclusion:* $(p_i, op_i) \in \lambda_i$.

(2) *Containment comparability:* $\lambda_i \subseteq \lambda_j \vee \lambda_j \subseteq \lambda_i$.
(3) *Process sequentiality: if* $p_i = p_j \wedge op_i \neq op_j$, *then* $(p_i, op_i) \notin \lambda_j \vee (p_j, op_j) \notin \lambda_i$.

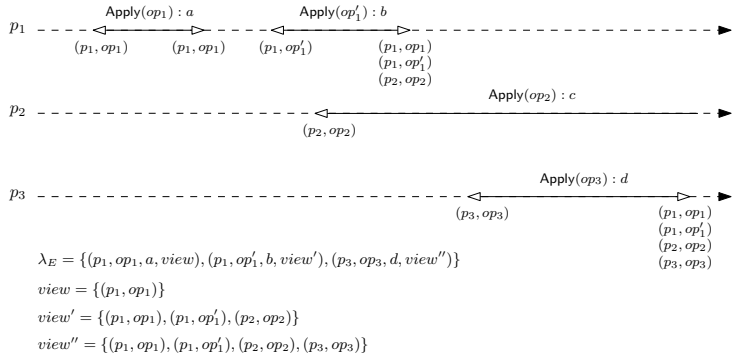


Fig. 9. The figure shows the history of a tight execution E of $\mathcal{A}^*$. The invocation pair and the view of each operation is depicted. The set λ_E is depicted too. It is easy to check that the construction from λ_E produces a history $X(\lambda_E)$ that is equivalent to E with $\prec_E = \prec_{X(\lambda_E)}$, and hence E and $X(\lambda_E)$ are similar to each other.

In the rest of this subsection, our discussion is focused on the tight executions of $\mathcal{A}^*$, particularly, in showing how the histories associated to them can be obtained from their views.

For any tight execution E of $\mathcal{A}^*$, let λ_E denote the set with all 4-tuples $(p_i, op_i, y_i, \lambda_i)$ such that (y_i, λ_i) is the response of operation Apply(op_i) of p_i in E (see Figure 9). We now explain that a well-formed history $X(\lambda_E)$ can be obtained from λ_E, and explain in what sense $X(\lambda_E)$ is a sketch of E. The construction that follows is from [17].

By Remark 7.2 (2), all *distinct* views that appear in λ_E can be ordered in strictly containment ascending order: $\sigma_1 \subset \sigma_2 \subset \ldots \subset \sigma_m$. Let σ_0 denote $\emptyset$. For each $k = 1, 2, \ldots, m$, $X(\lambda_E)$ is iteratively obtained following the next two steps in ascending order (see Figure 9 for an example):

(1) For each invocation pair $(p_i, op_i) \in \sigma_k \setminus \sigma_{k-1}$, the invocation to operation Apply(op_i) by p_i is appended to $X(\lambda_E)$; the invocations are appended in any arbitrary order.
(2) For each $(p_i, op_i, y_i, \lambda_i) \in \lambda_E$ with $\lambda_i = \sigma_k$, the response from operation Apply(op_i) with output (y_i, λ_i) by p_i is appended to $X(\lambda_E)$; the responses are appended in any arbitrary order.

In each of the steps of the construction above, either a set of invocations or responses are placed in some arbitrary sequential order. For any of these orders, the resulting history has the same relation $\prec$ over pending and complete operations, by construction, and hence all possible histories obtained in this way are similar to one another. Thus, in fact, $X(\lambda_E)$ denotes an *equivalence class* of histories.[5] By similarity-closure of GenLin, we have:

Claim 7.1. *For every object O in* GenLin *and every tight execution E of $\mathcal{A}^*$, either all histories of $X(\lambda_E)$ are in O or none of them is in O.*

By abuse of notation, we let $X(\lambda_E)$ denote any history of the equivalence class, unless stated otherwise.

[5]In the parlance of [17], $X(\lambda_E)$ is an *interval-sequential history*, i.e., an alternating sequence of non-empty sets with only either invocations or responses, starting with a set of invocations. Interval-sequential histories are used in [17] to define interval-linearizability, a generalization of linearizability in which, roughly speaking, an operation is allowed to overlap other operations in an interval-linearization of an execution.

The duality between histories and sets of views has been investigated in [17], where it is shown that the construction above is a bijection between (equivalence classes of) well-formed finite histories and sets of 4-tuples $(p_i, op_i, y_i, \lambda_i)$ whose views satisfy the properties in Remark 7.2. The views can be understood as a *static* mechanism, made of unordered sets, that fully capture the dynamic real-time order of operations in a history. Lemma 7.1 in [17] directly implies that $X(\lambda_E)$ is indeed an accurate sketch of E in the following sense:

Lemma 7.4 (Views are sketches of tight executions). *For any tight execution E of $\mathcal{A}^*$, E and $X(\lambda_E)$ are equivalent with $\prec_E = \prec_{X(\lambda_E)}$, hence the histories are similar to one another.*

As every $O \in$ GenLin is closed by similarity, we have:

Corollary 7.1. *For any tight execution E of $\mathcal{A}^*$ and every object O in* GenLin*,*

$$E \in O \iff X(\lambda_E) \in O.$$

Claim 7.2. *If H is a history of an implementation $\mathcal{B}$ (not necessarily in the class* DRV*), then any history H' that is equivalent to H with $\prec_H = \prec_{H'}$ is a history of $\mathcal{B}$ as well.*

Proof. Since H is a history of $\mathcal{B}$, there is an execution E of $\mathcal{B}$ such that H is the history obtained from E. History H has the form $I_1R_1I_2R_2\ldots$, where each I_x (resp. R_x) is a non-empty sequence of invocations (resp. responses); note that the invocations in I_x (resp. responses in R_x) does not necessarily appear in a continuos sequence in E. The main observation to prove the claim is that for every history H' such that is equivalent to H with $\prec_H = \prec_{H'}$, we must have that $H' = I'_1R'_1I'_2R'_2\ldots$ with I'_x (resp R'_x) being a permutation of I_x (resp. R_x). Then, consider the execution E' obtained from E as follows: (1) for each I_x, first move all invocations in I_x to the position of the *first* invocation in I_x, and then permute them according to I'_x, and similarly (2) for each R_x, first move all responses in R_x to the position of the *last* response in R_x, and then permute them according to R'_x. Observe that E' is an execution of $\mathcal{B}$ because only invocation and responses of E, which are local steps, are modified to obtain E', and it is immaterial when these local steps actually occur, as long as the specification of $\mathcal{B}$ is satisfied, as it happens in E'. By construction, H' is the history obtained from E', and therefore, H' is a history of $\mathcal{B}$. □

Finally, Lemma 7.4 and Claim 7.2 imply:

Corollary 7.2. *For any tight execution E of $\mathcal{A}^*$, $X(\lambda_E)$ is a history of $\mathcal{A}^*$.*

We stress that all claims in this subsubsection but Claim 7.2 focus on the tight executions of $\mathcal{A}^*$. In fact, Corollary 7.1 is not true if E is not tight, as the example at the top of Figure 6 shows (E is linearizable but $X(\lambda_E)$, which corresponds to E' in the example, is not).

7.4 A note on the class DRV and refined task solvability

The views mechanism was introduced in [17] (implicitly defined in [45] too) in order to extend the task specification formalism [60] to make it able to capture linearizable sequential long-lived objects. The resulting formalism is called multi-shot refined tasks, where processes are required to produce outputs and *implicitly* produce views. It turns out that multi-shot refined tasks are strictly more expressive than linearizability, and are equally expressive as interval-linearizability, also introduced in [17]: for every interval-sequential object, there is an equivalent multi-shot refined task, and vice versa. From this perspective, the implementations in DRV solve the corresponding equivalent multi-shot refined tasks, producing *explicit* views, which is what makes them predictively runtime verifiable.

8 PREDICTIVE RUNTIME VERIFIABILITY OF DRV AND SELF-ENFORCED GENLIN IMPLEMENTATIONS

8.1 A wait-free predictive verifier for DRV

Lemmas 7.3 and 7.4 in the previous section are the basis of the wait-free predictive verifier $\mathcal{V}_O$ in Figure 10. The idea of the verifier is simple. For any finite execution E of the verifier, the views in $\lambda_{T(E|\mathcal{A}^*)}$ sketch the tight execution $T(E|\mathcal{A}^*)$ associated to the current execution $E|\mathcal{A}^*$ of $\mathcal{A}^*$ (Lemma 7.4), and tight executions suffice to fulfill predictive soundness and completeness (Lemma 7.3). Thus, in $\mathcal{V}_O$ the processes simply exchange their views using Write and Snapshot (Lines 07 to 09) and then each process locally tests if the execution it reads from the shared memory is correct (Lines 10 to 12).

```
Shared Variables:
  M = wait-free linearizable snapshot object, with each entry initialized to ∅

Operation Verify(A* ∈ DRV) is
(01) res_i ← ∅
(02) while true do
(03)     op_i ← non-deterministically chosen operation that does not appear in res_i
(04)     Invoke operation Apply(op_i) of A*
(05)     (y_i, λ_i) ← response from operation Apply(op_i) of A*
(06)     res_i ← res_i ∪ {(p_i, op_i, y_i, λ_i)}
(07)     M.Write(res_i)
(08)     s_i ← M.Snapshot()
(09)     τ_i ← ⋃_{k∈{1,2,...,n}} s_i[k]
(10)     if X(τ_i) ∉ O then
(11)         report (ERROR, X(τ_i))
(12)     end if
(13) end while
end Verify
```

Fig. 10. A wait-free predictive verifier $\mathcal{V}_O$ with linear step complexity, for any object $O \in$ GenLin and any implementation $\mathcal{A}^* \in$ DRV (code of process p_i).

Although $\mathcal{V}_O$ relies on a simple idea, proving it correct is not simple at all. The main reason is that in its Snapshot step in Line 08, a process might obtain only a proper subset of $\lambda_{T(E|\mathcal{A}^*)}$, hence the history $X(\tau_i)$, locally constructed as explained in Section 7.3.3, used in the test in Line 10 might not be exactly $T(E|\mathcal{A}^*)$, the sketch of the current history of $\mathcal{A}^*$. This situation might happen due to asynchrony, as some processes might have already obtained a response from $\mathcal{A}^*$ in $E|\mathcal{A}^*$ (Line 05) but no written yet their responses in M (Line 07). Namely, processes might make decisions (Line 10) with incomplete information. The main issue one needs to resolve when proving the correctness of $\mathcal{V}_O$ is that processes make consistent decisions, despite the possible lack of information. To deal with this issue, Lemmas 8.1 and 8.2 below show useful properties of the histories computed in Line 10.

As mentioned in the previous section, the Snapshot object can be wait-free linearizable implemented using only Read/Write base objects, and moreover, there are implementations with $O(n)$ step complexity [63]. Thus, clearly $\mathcal{V}_O$ is a wait-free, and uses only Read/Write base objects, and every iteration of the while loop takes $O(n)$ steps. Thus we have:

Claim 8.1. *The verifier $\mathcal{V}_O$ in Figure 10 is wait-free, uses only* Read/Write *base objects with step complexity $O(n)$.*

For proving correctness of $\mathcal{V}_O$, we analyze only its infinite executions in which the sequence of *local* computation steps in Lines 09 to 12 of a process p_i appear all together right after the previous Snapshot step of p_i in Line 08 (which is part of the same loop iteration). Restricting our attention

to these executions facilitates the discussion and proves the verifier to be correct in all cases, as it is immaterial when these local steps actually occur. This restriction can also be seen from a slightly different perspective: given any infinite execution of the verifier, the local steps in Lines 09 to 12 of a process p_i can be moved backwards to be right next to the previous Snapshot step of p_i in Line 08, and the processes still make the same decisions in the modified execution.

Lemma 8.1. *Let E be any infinite execution of the wait-free verifier $\mathcal{V}_O$ in Figure 10 with an arbitrary input $\mathcal{A}^* \in$ DRV. Consider any finite prefix E' of E whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i, and let τ' denote the content of τ_i of p_i at the end of E'. Then:*

(1) *$X(\tau')$ is a history of $\mathcal{A}^*$;*
(2) *for every object O in* GenLin, *$T(E'|\mathcal{A}^*) \in O \implies X(\tau') \in O$.*

Proof. By Lemma 7.3, $T(E'|\mathcal{A}^*)$ is an execution of $\mathcal{A}^*$. By analyzing $T(E'|\mathcal{A}^*)$, we will conclude that $X(\tau')$ is a history of $\mathcal{A}^*$. By the definition of E', it follows that τ' contains all 4-tuples that appear in M at the end of E'. Due to asynchrony, it is possible that not all 4-tuples in $\lambda_{T(E'|\mathcal{A}^*)}$ appear in M at the end of E'; the reason is that it can be the case that in E' a process executes its Snapshot in Line 05 of $\mathcal{A}^*$, and hence the 4-tuple of the corresponding operation is in $T(E'|\mathcal{A}^*)$, by definition of tight executions, but it does not execute its Write step in Line 07 of $\mathcal{V}_O$. Hence, at the end of E', the response of the last operation in $T(E'|\mathcal{A}^*)$ of a process in might be "missing" in τ'. We thus have that $\tau' \subseteq \lambda_{T(E'|\mathcal{A}^*)}$; moreover, $\lambda_{T(E'|\mathcal{A}^*)} \setminus \tau'$ has at most one 4-tuple for each process.

We will modify $T(E'|\mathcal{A}^*)$ to obtain an execution F of $\mathcal{A}^*$ whose history is precisely $X(\tau')$. As already said, each 4-tuple that appears in $\lambda_{T(E'|\mathcal{A}^*)} \setminus \tau'$ corresponds to an operation that is complete in $T(E'|\mathcal{A}^*)$ and that operation is the last one of the corresponding process in the execution. Let F be the execution obtained from $T(E'|\mathcal{A}^*)$ as follows: for each 4-tuple in $\lambda_{T(E'|\mathcal{A}^*)} \setminus \tau'$, remove the steps in Lines 05 to 07 of $\mathcal{A}^*$ (see Figure 7) of the operation corresponding to the 4-tuple. Clearly, F is an execution of $\mathcal{A}^*$ as only the last steps and response of the last operation of some processes are removed from $T(E'|\mathcal{A}^*)$. Moreover, it is a tight execution as each of its pending operations does not execute the Snapshot step in Line 05 of $\mathcal{A}^*$. By construction, we have that $\tau' = \lambda_F$, from which follows that $X(\tau')$ and $X(\lambda_F)$ are equivalent with $\prec_{X(\tau')} = \prec_{X(\lambda_F)}$. Finally, F and $X(\lambda_F)$ are equivalent with $\prec_F = \prec_{X(\lambda_F)}$, by Lemma 7.4, and hence $X(\tau')$ is a history of $\mathcal{A}^*$, by Claim 7.2 and as F is an execution of $\mathcal{A}^*$.

From the previous discussion we can see that F is similar to $T(E'|\mathcal{A}^*)$. Let F' be the history obtained from F by appending the responses in the 4-tuples of $\lambda_{T(E'|\mathcal{A}^*)} \setminus \tau'$. Observe that F' and $T(E'|\mathcal{A}^*)$ are equivalent, and $\prec_{F'} = \prec_{T(E'|\mathcal{A}^*)}$. Since any object $O \in$ GenLin is closed by similarity, we conclude that $T(E'|\mathcal{A}^*) \in O \implies F \in O$. From the discussion above, we know that F and $X(\tau')$ are similar to one another, and hence $F \in O \iff X(\tau')$, and therefore $T(E'|\mathcal{A}^*) \in O \implies X(\tau') \in O$. □

Lemma 8.2. *Let E be any infinite execution of the wait-free verifier $\mathcal{V}_O$ in Figure 10 with an arbitrary input $\mathcal{A}^* \in$ DRV. Consider any finite prefix E' of E. There is a finite prefix F of E such that for every finite prefix E'' of E that has F as one its prefixes and whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i, it holds that $T(E'|\mathcal{A}^*)$ is similar to a prefix of $X(\tau'')$, where τ'' denotes the content of τ_i of p_i at the end of E''.*

Proof. Lemma 7.4 implies that we can concentrate on $X(\lambda_{T(E'|\mathcal{A}^*)})$ to reason about $T(E'|\mathcal{A}^*)$. Observe that due to asynchrony, it is possible that not all 4-tuples in $\lambda_{T(E'|\mathcal{A}^*)}$ appear in M at the end of E'; the reason is that it is possible that in E' a process executes its Snapshot in Line 05 of $\mathcal{A}^*$ (see Figure 7), and hence the 4-tuple of the corresponding operation is in $\lambda_{T(E'|\mathcal{A}^*)}$, by definition of tight executions, but it does not execute its Write step in Line 07 of $\mathcal{V}_O$. Thus, at the end of E', the response of the last operation in $\lambda_{T(E'|\mathcal{A}^*)}$ of a process might be "missing" in M. Since E is

infinite and by assumption fair (see Section 2), there is a finite prefix F of it in which all 4-tuples in $\lambda_{T(E'|\mathcal{A}^*)}$ appear in M at the end of F; note that at the of F, M might contain 4-tuples of operations that do not appear in $\lambda_{T(E'|\mathcal{A}^*)}$. We claim that F is the prefix of E with the desired property.

Let E'' be any finite prefix of E that has F as one its prefixes and whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i, and let τ'' denotes the content of τ_i of p_i at the end of E''. Note that E' is a prefix of F. Using similar arguments as in the proof of Lemma 8.1, it can be argued that $\tau'' \subseteq \lambda_{T(E''|\mathcal{A}^*)}$. Moreover, the election of F and the definition of tight execution give that $\lambda_{T(E'|\mathcal{A}^*)} \subseteq \tau''$. It directly follows from the definition of tight execution that if an operation is pending in $T(E'|\mathcal{A}^*)$, then in E' it does not execute its Snapshot step in Line 05 of $\mathcal{A}^*$ (see Figure 7). We thus have that the view of any 4-tuple in $\tau'' \setminus \lambda_{T(E'|\mathcal{A}^*)}$ contains the largest view among the views in the 4-tuples in $\lambda_{T(E'|\mathcal{A}^*)}$.

We will now reason how $X(\lambda_{T(E'|\mathcal{A}^*)})$ and $X(\tau'')$ are constructed from $\lambda_{T(E'|\mathcal{A}^*)}$ and $\lambda_{\tau''}$, respectively. Let $\sigma'_1 \subset \sigma'_2 \subset \ldots \subset \sigma'_{k'}$ and $\sigma''_1 \subset \sigma''_2 \subset \ldots \subset \sigma''_{k''}$ be respectively the distinct views in $\lambda_{T(E'|\mathcal{A}^*)}$ and τ'', ordered in ascending order by containement. For the reasons above exposed, we have that (1) $k' \leq k''$, (2) $\sigma'_\ell = \sigma''_\ell$, for $1 \leq \ell \leq k'$, and (3) $\sigma'_{k'} \subset \sigma''_\ell$, for $k' < \ell \leq k''$. Therefore, the construction of $X(\lambda_{T(E'|\mathcal{A}^*)})$ and $X(\tau'')$ from $\lambda_{T(E'|\mathcal{A}^*)}$ and τ'' use the same first k' views. Let S be the subset of τ'' with all 4-tuples whose views are subset of $\sigma'_{k'}$. Note that $\lambda_{T(E'|\mathcal{A}^*)} \subseteq S$; intuitively, the difference between S and $\lambda_{T(E'|\mathcal{A}^*)}$ is that $\lambda_{T(E'|\mathcal{A}^*)}$ might be missing the response of the last operation of some processes and the views of these operation are contained by $\sigma'_{k'}$. Hence, $S \setminus \lambda_{T(E'|\mathcal{A}^*)}$ has at most one 4-tuple for each process. Let H be the shortest prefix of $X(\lambda_{T(E'|\mathcal{A}^*)})$ containing all operations whose views are subset of $\sigma'_{k'}$. From the definition of S and the construction of $X(S)$, it directly follows that H and $X(S)$ are similar to each other. To conclude the proof, we argue that $X(\lambda_{T(E'|\mathcal{A}^*)})$ is similar to H. Let X' be the history obtained by appending to $X(\lambda_{T(E'|\mathcal{A}^*)})$ the responses in the 4-tuples of $S \setminus \lambda_{T(E'|\mathcal{A}^*)}$. From the discussion above, we can conclude that X' and H are equivalent and $\prec_{X'} = \prec_H$. Therefore, $X(\lambda_{T(E'|\mathcal{A}^*)})$ is similar to H, and hence $T(E'|\mathcal{A}^*)$ is similar to H too. □

We are finally ready to prove that GenLin is predictively verifiable, with respect to the class DRV of implementations.

Theorem 8.1 (GenLin is runtime predictive verifiable with respect to the class DRV). *Let O be any object in the class* GenLin. *The verifier $\mathcal{V}_O$ in Figure 10 is a wait-free predictive verifier for the correctness of O for the class* DRV *of implementations. Furthermore, $\mathcal{V}_O$ satisfies the following properties.*

(1) *Efficiency. It uses only* Read/Write *base objects with step complexity $O(n)$.*
(2) *Soundness for correct executions of $\mathcal{A}$. In each infinite execution E of it with input $\mathcal{A}^*$, if for every finite prefix E' of E, it holds that $E'|\mathcal{A} \in O$ (i.e. the history of $\mathcal{A}$, the underlying implementation of $\mathcal{A}^*$, is correct), then no process reports* ERROR *in E.*
(3) *Stability. For every infinite execution E of it in which at least one process reports* ERROR*, there is a finite prefix of it such that it is reported* ERROR *in every new loop iteration starting after the prefix.*

Proof. As it is shown in Claim 8.1, the verifier $\mathcal{V}_O$ is wait-free, uses only Read/Write base objects with step complexity $O(n)$, and hence it satisfies the efficiency property.

We now argue that $\mathcal{V}_O$ is a predictive verifier for the correctness of O, namely, it satisfies predictive soundness and completeness; we also argue that it satisfies soundness for correct executions of $\mathcal{A}$ and stability. Consider any infinite execution E of $\mathcal{V}_O$ with an arbitrary input implementation $\mathcal{A}^* \in$ DRV.

- Predictive soundness. Suppose that there is a finite prefix E' of E whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i, and p_i reports ERROR at the end of E'. Clearly, p_i reports ERROR because $X(\tau') \notin O$, where τ' denotes the content of τ_i of p_i at the end of E'; hence p_i reports the history $X(\tau')$. By Lemma 8.1 (1), $X(\tau')$ is a history of $\mathcal{A}^*$, and hence is a witness for $\mathcal{A}^*$. Thus, predictive soundness is satisfied.
- Soundness for correct executions of $\mathcal{A}$. Consider any finite prefix E'' of E, whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i. By Lemmas 7.3 and 8.1 (2), $E''|\mathcal{A} \in O \implies T(E''|\mathcal{A}^*) \in O \implies X(\tau'') \in O$, where τ'' denotes the content of τ_i of p_i at the end of E''. Thus, p_i does not report ERROR at the end of E''.
- Completeness and stability. Consider any finite prefix E' of E such that $E'|\mathcal{A}^* \notin O$. By Lemma 7.3, $E'|\mathcal{A}^* \notin O \implies T(E'|\mathcal{A}^*) \notin O$. Lemma 8.2 guarantees the existence of a finite prefix F of E such that for every finite prefix E'' of E that has F as one its prefixes and whose last sequence of steps correspond to the steps in Lines 08 to 12 of a process p_i, it holds that $T(E'|\mathcal{A}^*)$ is similar to a prefix H of $X(\tau'')$, where τ'' denotes the content of τ_i of p_i at the end of E''. Since O is closed by similarity, we have $T(E'|\mathcal{A}^*) \notin O \implies H \notin O$, and since it is closed by prefixes, $H \notin O \implies X(\tau'') \notin O$. Thus we conclude that p_i reports $(\text{ERROR}, X(\tau''))$ at the end of E''. By Lemma 8.1 (1), $X(\tau'')$ is a history of $\mathcal{A}^*$, and hence is a witness for $\mathcal{A}^*$. Therefore, completeness is satisfied. Observe that stability is satisfied too as the analysis holds for every such E'' having F as a prefix.

□

8.2 Runtime self-enforced correct implementations for GenLin

From Theorem 8.1, we can obtain a simple and generic methodology that, starting with an arbitrary GenLin implementation $\mathcal{A}$, produces a GenLin implementation $\mathcal{B}$ whose responses are runtime verified. We call the implementations obtained with this methodology *self-enforced* GenLin. The methodology is as follows.

```
Shared Variables:
  M = wait-free linearizable snapshot object, with each entry initialized to ∅

Local Persistent Variable:
  set_i = a set initialized to ∅

Operation Apply(op_i) is
(01) Invoke operation Apply(op_i) of A*
(02) (y_i, λ_i) ← response from operation Apply(op_i) of A*
(03) res_i ← res_i ∪ {(p_i, op_i, y_i, λ_i)}
(04) M.Write(res_i)
(05) s_i ← M.Snapshot()
(06) τ_i ← ⋃_{k∈{1,2,...,n}} s_i[k]
(07) if X(τ_i) ∈ O then
(08)     return y_i
(09) else
(10)     return (ERROR, X(τ_i))
(11) end if
end Apply
```

Fig. 11. Self-enforced GenLin implementation $\mathcal{V}_{O,\mathcal{A}}$ (code of process p_i).

Let $\mathcal{A}$ be any implementation for an object $O \in$ GenLin. Consider the DRV implementation $\mathcal{A}^*$ obtained from $\mathcal{A}$ (Figure 7), and the predictive verifier $\mathcal{V}_O$ for O (Figure 10). Let $\mathcal{V}_{O,\mathcal{A}}$ be the implementation obtained from $\mathcal{V}_O$ and $\mathcal{A}^*$ as shown in Figure 11: its high-level operation $\text{Apply}(op_i)$ executes Lines 04 to 09 of $\mathcal{V}_O$ with $\mathcal{A}^*$, and returns $resp_i$ if $X(\tau_i) \in O$, and $(\text{ERROR}, X(\tau_i))$

otherwise. We stress that the methodology is generic, as $\mathcal{V}_{O,\mathcal{A}}$ does not rely on any specific property of $\mathcal{A}$.

Theorem 8.2 below shows that indeed $\mathcal{V}_{O,\mathcal{A}}$ self-enforces GenLin correctness. Furthermore, at any time $\mathcal{V}_{O,\mathcal{A}}$ is able to produce a history that *certifies* that its responses so far are correct or incorrect.

Theorem 8.2 (Runtime self-enforced correct implementations for GenLin). *Let O be any object in* GenLin *and $\mathcal{A}$ be* any *implementation. Consider the implementation $\mathcal{V}_{O,\mathcal{A}}$ obtained from $\mathcal{A}$ as specified in Figure 11. Then,*

(1) *$\mathcal{V}_{O,\mathcal{A}}$ and $\mathcal{A}$ have the same progress condition,*
(2) *if $\mathcal{A}$ is correct with respect to O, then $\mathcal{V}_{O,\mathcal{A}}$ is correct with respect to O; otherwise every finite execution of $\mathcal{V}_{O,\mathcal{A}}$ is correct with respect to O up to a prefix (which does not necessarily exist) where every new operation returns* ERROR *together with a witness for $\mathcal{A}^*$,*
(3) *if requested, $\mathcal{V}_{O,\mathcal{A}}$ can output a history that is similar to the history of $\mathcal{V}_{O,\mathcal{A}}$ at the moment of the request.*

Proof. First, since $\mathcal{V}_O$ is wait-free, by Theorem 8.1, and $\mathcal{A}$ and $\mathcal{A}^*$ have the same progress condition, by Lemma 7.2, we have that $\mathcal{V}_{O,\mathcal{A}}$ and $\mathcal{A}$ have the same progress condition.

Observe that, by the definition of $\mathcal{V}_{O,\mathcal{A}}$, every infinite execution E of $\mathcal{V}_O(\mathcal{A}^*)$ is naturally mapped to a *unique* infinite execution E' of $\mathcal{V}_{O,\mathcal{A}}$, where the histories of $\mathcal{A}$ and $\mathcal{A}^*$ are exactly the same in both executions, and every process passes through essentially the same sequence of local states. Basically, E' is obtained from E by respectively replacing the beginning and end of a loop iteration with the invocation and response of the operation invoked in the iteration.

Consider the case where $\mathcal{A}$ is correct with respect to O, and let E be any infinite execution of $\mathcal{V}_{O,\mathcal{A}}$. The soundness for correct executions of $\mathcal{A}$ property of $\mathcal{V}_O$ (see Theorem 8.1) implies that no process reports ERROR in E. Thus, every operation returns the same response in $E|\mathcal{A}$ and E. Consider any finite prefix E' of E. Using a similar reasoning as in previous proofs, it can be shown that E' is similar to $E'|\mathcal{A}$, and hence $E'|\mathcal{A} \in O \implies E' \in O$, as O is closed by similarity. Therefore, $\mathcal{V}_{O,\mathcal{A}}$ is correct with respect to O, as $\mathcal{A}$ is correct with respect to O.

Suppose now that $\mathcal{A}$ is not correct with respect to O, and let E be any infinite execution of $\mathcal{V}_{O,\mathcal{A}}$. Consider any finite prefix E' of E. The argument in the previous paragraph shows that if $E'|\mathcal{A} \in O$, then no process reports ERROR in E', and hence $E' \in O$. Thus consider the case $E'|\mathcal{A} \notin O$. The completeness and stability properties of $\mathcal{V}_O$ (see Theorem 8.1) implies that eventually all new operations in E report ERROR together with a witness X for $\mathcal{A}^*$. To conclude the proof, for the sake of contradiction, suppose that no process reports ERROR in E' but $E' \notin O$. Using a similar reasoning as in previous proofs, it can be shown that E' is similar to $T(E'|\mathcal{A}^*)$, and hence $E' \notin O \implies T(E'|\mathcal{A}^*) \notin O$, as O is closed by similarity. Since no process reports ERROR in E', all invocation an responses that appear in it have been written in the shared memory M at the end of E'. Thus the last process that takes its Snapshot in Line 08 of $\mathcal{V}_O$ reads the views of all these operations, and hence the history it computes in Line 08 is $T(E'|\mathcal{A}^*)$, from which follows that it reports ERROR in E', as we already saw that $T(E'|\mathcal{A}^*) \notin O$. We have reached a contradiction.

Finally, at any time a process can snapshot from the shared memory of $\mathcal{V}_O$ a history that is similar to $T(E'|\mathcal{A}^*)$, which can be returned if requested through Apply. □

At first glance, one might think that the self-enforced GenLin implementation $\mathcal{V}_{O,\mathcal{A}}$ in Theorem 8.2 is somehow runtime verifying $\mathcal{A}$, hence contradicting the impossibility in Theorem 5.1; there is no contradiction however. As $\mathcal{A}^*$ is able to enforce correctness in only *some* incorrect executions of $\mathcal{A}$ (see Sections 6 and 7), it is possible that there are executions of $\mathcal{V}_{O,\mathcal{A}}$ whose

execution of $\mathcal{A}$ is not correct, but no process reports ERROR, which happens because $\mathcal{A}^*$ is able to fix that execution of $\mathcal{A}$, i.e., it enforces a correct behavior.

We conclude this subsection by observing that constructing $X(\tau_i)$ in Line 07 of $\mathcal{V}_{O,\mathcal{A}}$ can be locally computed in polynomial time in the number of operations in τ_i (see the construction in Section 7.3.3). Thus, the local test in that line is computed in the time it takes to test if $X(\tau_i)$ is in O, plus the polynomial overhead incurred to build $X(\tau_i)$. This polynomial overhead is desirable for linearizability since it is known that, for some sequential objects, linearizability of an execution can be decided in polynomial time [15, 32].

8.3 Concurrent systems with accountable and forensic guarantees

Consider a situation where a concurrent algorithm C uses an implementation $\mathcal{A}$ of an object $O \in$ GenLin. Theorem 8.2 show that, in C, $\mathcal{A}$ can be replaced with its self-enforced GenLin version $\mathcal{V}_{O,\mathcal{A}}$, whose non-ERROR responses are runtime verified. Moreover, if $\mathcal{A}$ is actually correct, then C never obtains ERROR from $\mathcal{V}_{O,\mathcal{A}}$, hence C cannot distinguish it is using $\mathcal{V}_{O,\mathcal{A}}$ instead of $\mathcal{A}$. This idea can be extended to all GenLin implementations used in C. If all those implementations are replaced with their self-enforced GenLin versions, the modified algorithm C is enriched with a mechanism from which incorrect GenLin implementation can be accounted, and witnesses histories of the self-enforced GenLin implementations can potentially be used in a forensic stage, which C could trigger once an incorrect response is detected at runtime.

9 EXTENSIONS

9.1 Base objects of bounded size

The implementation $\mathcal{A}^*$ (Figure 7) and the verifier $\mathcal{V}_O$ (Figure 10) use shared object of unbounded size. This unrealistic assumption can be removed by representing sets as linked lists. For $\mathcal{A}^*$, each entry in N contains the first node of a single linked list with the items in the set of the process associated to that entry. Every time a process adds an item to its set, it creates a new node, links it to the first node of the list (which then becomes the second node) and then writes the new node in its entry in N. A snapshot operation in $\mathcal{A}^*$ now returns a vector of positions in the linked lists, hence the elements in the sets in the snapshot are the nodes that are accesible from the positions. The sets in $\mathcal{V}_O$ can be represented in the same way.

9.2 Decoupled self-enforced GenLin implementations

In the self-enforced implementation $\mathcal{V}_{O,\mathcal{A}}$ (Figure 11), every process is in charge of producing responses and verifying them correct, which, from a performance perspective, might be undesirable. This issue can be mitigated by *decoupling* response production and verification. Namely, a group of processes $v_1, \ldots, v_m$, the *verifiers*, is in charge of the verification task, and the rest of the processes $p_1, \ldots, p_n$, the *producers*, are in charge of obtaining responses using $\mathcal{A}^*$, and store the sketch produced by $\mathcal{A}^*$ for the verifiers to perform the verification task. The decoupled version $\mathcal{D}_{O,\mathcal{A}}$ of $\mathcal{V}_{O,\mathcal{A}}$ appears in Figure 12. Observe that, in contrast to $\mathcal{V}_{O,\mathcal{A}}$, $\mathcal{D}_{O,\mathcal{A}}$ might produce incorrect responses (due to delays, at a given time verifiers might verify a response that was already returned), however, eventually the verifiers detect them, assuming that not all of them crash in an execution. In the terminology of [13], the producers implement the *communication interface* and the verifiers implement the *monitoring system*.

9.3 Verifying obstruction-free and blocking implementations, and task solvability

The proposed interactive model can be modified to include obstruction-free [61] or blocking [61] implementations. The difference is that the interaction between the implementation and a verifier

```
Shared Variables:
  M = wait-free linearizable snapshot object, with each entry initialized to ∅

Local Persistent Variable of Producer p_i:
  set_i = a set initialized to ∅

Operation Apply(op_i) is    %% Operation for producer p_i
(01) Invoke operation Apply(op_i) of A*
(02) (y_i, λ_i) ← response from operation Apply(op_i) of A*
(03) res_i ← res_i ∪ {(p_i, op_i, y_i, λ_i)}
(04) M.Write(res_i)
(05) return y_i
end Apply

Operation Verify() is    %% Operation for verifier v_j; starts at beginning of computation
(06) while true do
(07)     s_j ← M.Snapshot()
(08)     τ_j ← ⋃_{k∈{1,2,...,n}} s_j[k]
(09)     if X(τ_j) ∉ O then
(10)         report (ERROR, X(τ_j))
(11)     end if
(12) end while
end Verify
```

Fig. 12. Decoupled self-enforced GenLin implementation $\mathcal{D}_{O,\mathcal{A}}$ (codes of producer p_i and verifier v_j).

might be finite because the implementation might block; thus the required properties, predictive soundness and completeness, need to be satisfied even if the interaction is finite.

Similarly, task solvability can be predictively runtime verified. A task is a one-shot distributed problem [60]. It is known that any task can be equivalently modelled and a one-shot interval-sequential interval-linearizable object [17], which belongs to GenLin, since GenLin includes interval-linearizability. Again, the difference is that the interaction with this object is finite. Moreover, since every process invokes exactly one high-level operation, now a process can declare that the computation is *correct* if it reads from the shared memory the views of all processes in the system and the corresponding history is correct.

9.4 Extension to other models of computation

The proposed implementations assume a fixed number of processes that can participate in the computation. The implementations can however be adapted to shared memory models with an unbounded number of processes, i.e., where there is no prior knowledge on the number of processes that participate. We only have to use the wait-free snapshot implementation in [46] for this kind of models.

Due to the shared memory simulation algorithm in [5], all our algorithms, $\mathcal{A}^*$, $\mathcal{V}_O$, $\mathcal{V}_{O,\mathcal{A}}$ and $\mathcal{D}_{O,\mathcal{A}}$ can be simulated in asynchronous message-passing systems where less than half processes can crash in any execution.

10 RELATED WORK

Runtime verification. Runtime verification is an active field of research with important advancements in the last two decades. It has been employed in academia and industry for verification before system deployment in order to ensure correctness after deployment. Runtime verification has been used mostly to analyze software, however it has also been applied to other types of systems, e.g. hardware, hybrid and embedded systems, cyber-physical systems, distributed and concurrent systems, financial transaction systems, and more. For a detailed exposition of the field, we refer the reader to textbook [6] and surveys [36, 56, 71].

Distributed runtime verification. Distributed runtime verification of distributed systems is considered and emergent and important topic, that poses several challenges that are yet to be solved (see [13, 28, 33, 44, 72, 90]). Designing distributed, asynchronous, fault-tolerant communication interfaces are regarded as a difficult problem. As far as we know, there is no runtime verification algorithm in the literature (for any correctness property) that is at the same time fully asynchronous and fault-tolerant. There have been proposed distributed runtime verification algorithms for a number of properties that are failure-free synchronous message-passing (e.g. [4, 91]), fault-tolerant message-passing where processes have access to clocks that are synchronized at some level (e.g. [7, 9, 10, 29, 34, 49, 65, 88]), and fully asynchronous message-passing or shared memory with failure-free processes (e.g. [21, 38, 40, 73, 93, 96]).

Distributed fault-tolerant runtime verification. Our approach is close to the series of papers [12, 42, 43] initiated by Fraigniaud, Rajsbaum and Travers, who pioneered the study of distributed fault-tolerant verification [41]. In those works, it is studied a shared memory concurrent system that solves a series of tasks [60], and the aim is to runtime verify that the outputs for each task are correct, i.e. they satisfy the inputs/outputs relation specifying the task. To do so, an asynchronous wait-free read/write shared memory algorithm runs every time a task in the series is solved, and it is assumed that the verification algorithm terminates before the processes solve the next task. Thus, the distributed runtime verification algorithm proposed in those papers is distributed and fault-tolerant but it is *not* fully asynchronous. In sharp contrast, we consider a *full* asynchrony and wait-free shared memory system, where some processes might be verifying the current execution while at the same time others are executing a high-level operation. GenLin includes tasks, hence task solvability is covered by our results. The main concern in [12, 42, 43] is to understand how many distinct report values can be reported by the processes (*opinions* in the parlance of those papers) in order to runtime verify an algorithm. Our approach here is slightly different, as a process reports nothing (or implicitly makes a "so far so good" report) as long as the computation looks correct from its perspective, and, when a process "sees" enough information, it can decide whether the computation is correct or not. Another crucial difference is that the wait-free interactive verifier proposed here can detect real-time order violations (called validity for some problems) in a finer way, as real-time relations of the actual execution are taken into account, whereas in [12, 42, 43] real-time order of events is not considered. For example, by observing only (input,output) pairs, for consensus it is impossible to detect when a process ran solo and decided a value distinct from its input, which violates validity. That scenario, in contrast, can be detected by our verifier through the views mechanism of the class DRV.

Runtime verification of concurrent algorithms. For concurrent shared memory algorithms, runtime verification has been mostly used to detect data races, serializability violations (also called atomicity violations) and deadlocks; less studied properties are order instruction violations, missed signals, starvation, and high-level correctness conditions such as sequential consistency and linearizability (see [28, 72, 84]). Typically, in these works, asynchronous failure-free processes are assumed, and achieving a distributed communication interface is not a primary target. Several algorithms have been proposed for detecting data races and serializability violations; some algorithms use techniques based on the assumption that the concurrent algorithm under inspection uses locks (e.g. [92]), and others use some form of vector clocks to capture the happens-before relation, i.e. relations of causally-related events (e.g. [40]), or a mixture of both (e.g. [38]). None of these techniques can be adapted to our setting because: (1) we focus on lock-free implementations and (2) linearizability totally depends on the real-time order of non-causally related events. In general, a main difficulty is capturing the actual execution of a concurrent algorithm, as explained in [28, Section 4]. A simple solution is to serialize events using a lock (e.g. [77]); we find this type of solutions undesirable

because, first, it might change the progress condition of the algorithm under inspection (as explained in the Introduction), and second, the lock creates a bottleneck that compromises performance. Other algorithms rely on bytecode-level added instructions in order to capture the execution (e.g. [57]), and moreover there are dynamic analysis frameworks working at a bycodelevel that provide information of the current execution for performing dynamic analysis (e.g. [39]). A problem with these techniques is that the moment when an event happens and the moment when the event is registered are not the same (i.e. they do not occur atomically, as in our interactive model), and hence the actual execution might not be captured (which is the main argument in the proof of the impossibility in Theorem 5.1).

Runtime verification of linearizabilty. As far as we know, runtime verification of linearizability has only been studied in [30, 31], in asynchronous failure-free models with centralized monitoring systems. Those papers study I/O refinement, which generalizes linearizability for objects without sequential specifications. GenLin includes objects without sequential specifications too as it includes set-linearizability and interval-linearizability [17, 18, 81]. In [30, 31], specific code is added to a *white-box* implementation in order to record the execution in a *log* (a sequence of events) that later is verified by a single process (hence the runtime verification algorithm is neither distributed nor fault-tolerant). Events must be atomically recorded in the log, which necessarily requires consensus or the use of locks [61, 78, 85]. High-level operations are divided in *mutators* and *observers*. For mutators, the user has to add code that records in the log when the operation takes effect (i.e. its linearization point), and for observers, invocations and responses are recorded separately. It is known that there are linearizable implementations whose linearization points are *not fixed* [74], hence the approach in [30, 31] is not general. Finally, it is not explained in [30, 31] what the relation is between the actual execution of the algorithm under inspection and the execution recorded in the log.

Runtime enforcement. Runtime enforcement is an extension of runtime verification whose aim is to evaluate the current execution of a system under inspection, and halt the system whenever it deviates from a desired property (see survey [35]). Runtime enforcement initiated with the *security automata* formalism of Schneider [94]. Our interactive model for distributed runtime verification can be understood as a distributed version of Schneider's security automata. As far as we know, so far there have been proposed only a few distributed runtime enforcement algorithms (e.g. [8, 47, 50, 55, 89]).

Accountability. In general, accountability requires correct processes to *irrevocably* detect safety violations. Note that false positives are not allowed: once a violation is detected, the detection cannot be revoked. The concept of accountability in the context of distributed computing was introduced in [54]. Motivated by blockchain technologies, there have been recently proposed accountable algorithms for consensus [16, 22, 23, 97] and general tasks [24]. All these works consider semi-synchronous message-passing systems with malicious Byzantine failures, and the main target is to irrevocably detect Byzantine processes. Here we consider concurrent systems with benign crash failures, hence processes never deviate from its specification. In this scenario the safety violation one can detect are invalid outputs, as our self-linearizable implementations do.

11 FINAL DISCUSSION

This paper studied the problem of distributed runtime verification of linearizability in asynchronous wait-free shared memory systems, through a novel interactive model for runtime verification of correctness conditions. Distributed runtime verification of linearizability is *not* an agreement problem: regardless of the consensus number of the base objects used for verification, the problem is

impossible for common sequential objects such as queues, stacks, sets, priority queues, counters and the consensus problem. However, a predictive version of the problem can be solved for the class DRV of concurrent implementations, and without the need of consensus. Moreover, the possibility result holds for a correctness condition GenLin that includes linearizability and generalizations of it such as set-linearizability [81] and interval-linearizability [17, 18], the latter known to be expressive enough to model tasks [60] and any concurrent object satisfying some reasonable assumptions [17, 52]. GenLin contains any object that is closed by prefixes and similarity, the latter being a property identified here. Any concurrent implementation can be transformed into its counterpart in DRV, and there is a wait-free verifier that satisfies predictive soundness and completeness, for the class DRV and any object in GenLin. A crucial building block in the transformation to obtain DRV implementations is that of the views mechanism for capturing real-time order of executions [17]. Read/write objects suffice to solve predictive runtime verification, hence consensus among two or more processes is not needed.

A simple and generic methodology for designing self-enforced correct GenLin implementations was obtained. Given any concurrent implementation for some GenLin object, one can produce a self-enforced correct concurrent implementation with the same progress properties such that all outputs are runtime verified, or the implementation blocks, reporting error to every new invocation. These implementation are able to produce a certificate of the current computation at any time, hence allowing the design of systems in a modular manner with accountable and forensic guarantees. We are not aware of previous concurrent implementations in the literature with such properties. As far as we know, this is the first distributed runtime verification algorithm, for any correctness condition, that is at the same time fully asynchronous and fault-tolerant.

All together, the results show that runtime verification is possible if and only if the concurrent implementation under inspection outputs some information of its current history. The methodology for self-enforced correct implementations show how any GenLin implementation can be instrumented to make it runtime verifiable.

We believe several directions are worth to be explored. A natural direction is to study other correctness conditions in our interactive model such as sequential consistency [68] or causal consistency [3]. Runtime verification of hyperproperties [25] is interesting as well. The main algorithmic technique in our solutions is that of the views mechanism for sketching the current execution. In general it is interesting to explore if the mechanism helps to runtime verify other properties. In this direction, we have shown that the views mechanism allows predictive runtime verification of eventual consistency conditions such as strong eventual counters [20].

Finally, conducting experimental evaluations are important to understand if the proposed algorithms can provide good performance in practical settings. In follow-up work [87], we have improved the step complexity of $\mathcal{A}^*$ and $\mathcal{V}_O$ by removing the usage of snapshot objects. From the improved algorithms, it is derived a decoupled version of the self-enforced GenLin implementation (in the style of $\mathcal{D}_{O,\mathcal{A}}$ in Section 9.2), where, in order to produce a response, a producer executes the steps in $\mathcal{A}$ plus only *five* additional steps. The aim of those additional steps is capturing and storing the sketch of $\mathcal{A}^*$. It is yet to be empirically tested the performance of these improved implementations.

A LINEARIZABILITY FOR SOME OBJECTS IS NOT PREDICTIVELY VERIFIABLE

Theorem A.1 (Impossibility of Distributed Runtime PredictiveVerification). *Linearizability for queues, stacks, sets, priority queues, counters and the consensus problem (modeled as sequential objects) is not distributed runtime predictively verifiable, regardless of the consensus number of base objects used in a verifier.*

Proof. The proof is nearly the same as the proof of Theorem 5.1, where executions E and F of a hypothetical wait-free verifier $\mathcal{V}_{\text{queue}}$ are obtained from the non-linearizable queue implementation $\mathcal{A}$ defined in the proof. The difference between the proofs is in the last step, where it is now observed that the execution F can be equally obtained from any wait-free linearizable queue implementation $\mathcal{B}$ (several such implementations appear in [61]). The proof now concludes by observing that processes cannot report ERROR in F (which is allowed by predictive soundness) because there is no witness for $\mathcal{B}$ as it is linearizable. Thus, by indistinguishability, no process reports ERROR in E, and hence $\mathcal{V}_{\text{queue}}$ does not satisfy completeness. Therefore, $\mathcal{V}_{\text{queue}}$ cannot exist. □

Acknowledgements

We would like to thank Hagit Attiya, Gregory Chockler, Ori Lahav, Sergio Rajsbaum, David Rosenblueth, Serdar Tasiran and anonymous reviewers for insightful comments and discussions on this work. Part of this work was done while Armando Castañeda was on sabbatical leave visiting the Department of Computer Science of the University of Surrey. This work was partially supported by the research projects DGAPA-PAPIIT IN108723 and IN103126, and SECIHTI CBF-2025-I-393. Valeria Rodríguez-Jiménez is the recipient of a PhD fellowship from SECIHTI.

REFERENCES

[1] Y. Afek, H. Attiya, D. Dolev, E. Gafni, M. Merritt, and N. Shavit. Atomic snapshots of shared memory. *J. ACM*, 40(4):873–890, 1993.

[2] Y. Afek, G. Korland, and E. Yanovsky. Quasi-linearizability: Relaxed consistency for improved concurrency. In C. Lu, T. Masuzawa, and M. Mosbah, editors, *Principles of Distributed Systems - 14th International Conference, OPODIS 2010, Tozeur, Tunisia, December 14-17, 2010. Proceedings*, volume 6490 of *Lecture Notes in Computer Science*, pages 395–410. Springer, 2010.

[3] M. Ahamad, G. Neiger, J. E. Burns, P. Kohli, and P. W. Hutto. Causal memory: Definitions, implementation, and programming. *Distributed Comput.*, 9(1):37–49, 1995.

[4] H. Arfaoui, P. Fraigniaud, D. Ilcinkas, F. Mathieu, and A. Pelc. Deciding and verifying network properties locally with few output bits. *Distributed Comput.*, 33(2):169–187, 2020.

[5] H. Attiya, A. Bar-Noy, and D. Dolev. Sharing memory robustly in message-passing systems. *J. ACM*, 42(1):124–142, 1995.

[6] E. Bartocci and Y. Falcone, editors. *Lectures on Runtime Verification - Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*. Springer, 2018.

[7] D. A. Basin, M. Gras, S. Krstic, and J. Schneider. Scalable online monitoring of distributed systems. In J. Deshmukh and D. Nickovic, editors, *Runtime Verification - 20th International Conference, RV 2020, Los Angeles, CA, USA, October 6-9, 2020, Proceedings*, volume 12399 of *Lecture Notes in Computer Science*, pages 197–220. Springer, 2020.

[8] D. A. Basin, V. Jugé, F. Klaedtke, and E. Zalinescu. Enforceable security policies revisited. *ACM Trans. Inf. Syst. Secur.*, 16(1):3, 2013.

[9] D. A. Basin, F. Klaedtke, and E. Zalinescu. Failure-aware runtime verification of distributed systems. In P. Harsha and G. Ramalingam, editors, *35th IARCS Annual Conference on Foundation of Software Technology and Theoretical Computer Science, FSTTCS 2015, December 16-18, 2015, Bangalore, India*, volume 45 of *LIPIcs*, pages 590–603. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2015.

[10] A. Bauer and Y. Falcone. Decentralised LTL monitoring. *Formal Methods Syst. Des.*, 48(1-2):46–93, 2016.

[11] S. Berkovich, B. Bonakdarpour, and S. Fischmeister. Runtime verification with minimal intrusion through parallelism. *Formal Methods Syst. Des.*, 46(3):317–348, 2015.

[12] B. Bonakdarpour, P. Fraigniaud, S. Rajsbaum, D. A. Rosenblueth, and C. Travers. Decentralized asynchronous crash-resilient runtime verification. *J. ACM*, 69(5):34:1–34:31, 2022.

[13] B. Bonakdarpour, P. Fraigniaud, S. Rajsbaum, and C. Travers. Challenges in fault-tolerant distributed runtime verification. In T. Margaria and B. Steffen, editors, *Leveraging Applications of Formal Methods, Verification and Validation: Discussion, Dissemination, Applications - 7th International Symposium, ISoLA 2016, Imperial, Corfu, Greece, October 10-14, 2016, Proceedings, Part II*, volume 9953 of *Lecture Notes in Computer Science*, pages 363–370, 2016.

[14] A. Bouajjani, M. Emmi, C. Enea, and J. Hamza. Verifying concurrent programs against sequential specifications. In M. Felleisen and P. Gardner, editors, *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*, volume 7792 of *Lecture Notes in Computer Science*, pages 290–309. Springer, 2013.

[15] A. Bouajjani, M. Emmi, C. Enea, and J. Hamza. Tractable refinement checking for concurrent objects. In S. K. Rajamani and D. Walker, editors, *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, pages 651–662. ACM, 2015.

[16] V. Buterin and V. Griffith. Casper the friendly finality gadget. *CoRR*, abs/1710.09437, 2017.

[17] A. Castañeda, S. Rajsbaum, and M. Raynal. Unifying concurrent objects and distributed tasks: Interval-linearizability. *J. ACM*, 65(6):45:1–45:42, 2018.

[18] A. Castañeda, S. Rajsbaum, and M. Raynal. A linearizability-based hierarchy for concurrent specifications. *Commun. ACM*, 66(1):86–97, 2023.

[19] A. Castañeda and G. V. Rodríguez. Asynchronous wait-free runtime verification and enforcement of linearizability. In R. Oshman, A. Nolin, M. M. Halldórsson, and A. Balliu, editors, *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing, PODC 2023, Orlando, FL, USA, June 19-23, 2023*, pages 90–101. ACM, 2023.

[20] A. Castañeda and G. V. Rodríguez. Asynchronous fault-tolerant language decidability for runtime verification of distributed systems. In A. Balliu and F. Kuhn, editors, *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2025, Hotel Las Brisas Huatulco, Huatulco, Mexico, June 16-20, 2025*, pages 218–229. ACM, 2025.

[21] H. Chauhan, V. K. Garg, A. Natarajan, and N. Mittal. A distributed abstraction algorithm for online predicate detection. In *IEEE 32nd Symposium on Reliable Distributed Systems, SRDS 2013, Braga, Portugal, 1-3 October 2013*, pages 101–110. IEEE Computer Society, 2013.

[22] P. Civit, S. Gilbert, and V. Gramoli. Polygraph: Accountable byzantine agreement. In *41st IEEE International Conference on Distributed Computing Systems, ICDCS 2021, Washington DC, USA, July 7-10, 2021*, pages 403–413. IEEE, 2021.

[23] P. Civit, S. Gilbert, V. Gramoli, R. Guerraoui, and J. Komatovic. As easy as ABC: optimal (a)ccountable (b)yzantine (c)onsensus is easy! In *2022 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2022, Lyon, France, May 30 - June 3, 2022*, pages 560–570. IEEE, 2022.

[24] P. Civit, S. Gilbert, V. Gramoli, R. Guerraoui, J. Komatovic, Z. Milosevic, and A. Seredinschi. Crime and punishment in distributed byzantine decision tasks. In *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022*, pages 34–44. IEEE, 2022.

[25] M. R. Clarkson and F. B. Schneider. Hyperproperties. *J. Comput. Secur.*, 18(6):1157–1210, 2010.

[26] N. Decker, P. Gottschling, C. Hochberger, M. Leucker, T. Scheffel, M. Schmitz, and A. Weiss. Rapidly adjustable non-intrusive online monitoring for multi-core systems. In S. A. da Costa Cavalheiro and J. L. Fiadeiro, editors, *Formal Methods: Foundations and Applications - 20th Brazilian Symposium, SBMF 2017, Recife, Brazil, November 29 - December 1, 2017, Proceedings*, volume 10623 of *Lecture Notes in Computer Science*, pages 179–196. Springer, 2017.

[27] B. Dongol and J. Derrick. Verifying linearisability: A comparative survey. *ACM Comput. Surv.*, 48(2):19:1–19:43, 2015.

[28] A. El-Hokayem and Y. Falcone. Can we monitor all multithreaded programs? In C. Colombo and M. Leucker, editors, *Runtime Verification - 18th International Conference, RV 2018, Limassol, Cyprus, November 10-13, 2018, Proceedings*, volume 11237 of *Lecture Notes in Computer Science*, pages 64–89. Springer, 2018.

[29] A. El-Hokayem and Y. Falcone. On the monitoring of decentralized specifications: Semantics, properties, analysis, and simulation. *ACM Trans. Softw. Eng. Methodol.*, 29(1):1:1–1:57, 2020.

[30] T. Elmas and S. Tasiran. Vyrdmc: Driving runtime refinement checking with model checkers. In H. Barringer, B. Finkbeiner, Y. Gurevich, and H. Sipma, editors, *Proceedings of the Fifth Workshop on Runtime Verification, RV@CAV 2005, Edinburgh, UK, July 12, 2005*, volume 144 of *Electronic Notes in Theoretical Computer Science*, pages 41–56. Elsevier, 2005.

[31] T. Elmas, S. Tasiran, and S. Qadeer. VYRD: verifying concurrent programs by runtime refinement-violation detection. In V. Sarkar and M. W. Hall, editors, *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation, Chicago, IL, USA, June 12-15, 2005*, pages 27–37. ACM, 2005.

[32] M. Emmi and C. Enea. Sound, complete, and tractable linearizability monitoring for concurrent collections. *Proc. ACM Program. Lang.*, 2(POPL):25:1–25:27, 2018.

[33] Y. Falcone. On decentralized monitoring. In A. Nouri, W. Wu, K. Barkaoui, and Z. Li, editors, *Verification and Evaluation of Computer and Communication Systems - 15th International Conference, VECoS 2021, Virtual Event, November 22-23, 2021, Revised Selected Papers*, volume 13187 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2021.

[34] Y. Falcone, T. Cornebize, and J. Fernandez. Efficient and generalized decentralized monitoring of regular languages. In E. Ábrahám and C. Palamidessi, editors, *Formal Techniques for Distributed Objects, Components, and Systems - 34th IFIP WG 6.1 International Conference, FORTE 2014, Held as Part of the 9th International Federated Conference on Distributed Computing Techniques, DisCoTec 2014, Berlin, Germany, June 3-5, 2014. Proceedings*, volume 8461 of *Lecture Notes in Computer Science*, pages 66–83. Springer, 2014.

[35] Y. Falcone, J. Fernandez, and L. Mounier. What can you verify and enforce at runtime? *Int. J. Softw. Tools Technol. Transf.*, 14(3):349–382, 2012.

[36] Y. Falcone, K. Havelund, and G. Reger. A tutorial on runtime verification. In M. Broy, D. A. Peled, and G. Kalus, editors, *Engineering Dependable Software Systems*, volume 34 of *NATO Science for Peace and Security Series, D: Information and*

Communication Security, pages 141–175. IOS Press, 2013.

[37] M. J. Fischer, N. A. Lynch, and M. Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, 1985.

[38] C. Flanagan and S. N. Freund. Atomizer: A dynamic atomicity checker for multithreaded programs. *Sci. Comput. Program.*, 71(2):89–109, 2008.

[39] C. Flanagan and S. N. Freund. The roadrunner dynamic analysis framework for concurrent programs. In S. Lerner and A. Rountev, editors, *Proceedings of the 9th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering, PASTE'10, Toronto, Ontario, Canada, June 5-6, 2010*, pages 1–8. ACM, 2010.

[40] C. Flanagan, S. N. Freund, and J. Yi. Velodrome: a sound and complete dynamic atomicity checker for multithreaded programs. In R. Gupta and S. P. Amarasinghe, editors, *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, pages 293–303. ACM, 2008.

[41] P. Fraigniaud, S. Rajsbaum, and C. Travers. Locality and checkability in wait-free computing. In D. Peleg, editor, *Distributed Computing - 25th International Symposium, DISC 2011, Rome, Italy, September 20-22, 2011. Proceedings*, volume 6950 of *Lecture Notes in Computer Science*, pages 333–347. Springer, 2011.

[42] P. Fraigniaud, S. Rajsbaum, and C. Travers. Locality and checkability in wait-free computing. *Distributed Comput.*, 26(4):223–242, 2013.

[43] P. Fraigniaud, S. Rajsbaum, and C. Travers. A lower bound on the number of opinions needed for fault-tolerant decentralized run-time monitoring. *J. Appl. Comput. Topol.*, 4(1):141–179, 2020.

[44] A. Francalanza, J. A. Pérez, and C. Sánchez. Runtime verification for decentralised and distributed systems. In E. Bartocci and Y. Falcone, editors, *Lectures on Runtime Verification - Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 176–210. Springer, 2018.

[45] E. Gafni. Snapshot for time: The one-shot case. *CoRR*, abs/1408.3432, 2014.

[46] E. Gafni, M. Merritt, and G. Taubenfeld. The concurrency hierarchy, and algorithms for unbounded concurrency. In A. D. Kshemkalyani and N. Shavit, editors, *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing, PODC 2001, Newport, Rhode Island, USA, August 26-29, 2001*, pages 161–169. ACM, 2001.

[47] F. Gallay and Y. Falcone. Decentralized LTL enforcement. In P. Ganty and D. Bresolin, editors, *Proceedings 12th International Symposium on Games, Automata, Logics, and Formal Verification, GandALF 2021, Padua, Italy, 20-22 September 2021*, volume 346 of *EPTCS*, pages 135–151, 2021.

[48] R. Ganguly, A. Momtaz, and B. Bonakdarpour. Distributed runtime verification under partial synchrony. In Q. Bramas, R. Oshman, and P. Romano, editors, *24th International Conference on Principles of Distributed Systems, OPODIS 2020, December 14-16, 2020, Strasbourg, France (Virtual Conference)*, volume 184 of *LIPIcs*, pages 20:1–20:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[49] R. Ganguly, Y. Xue, A. Jonckheere, P. Ljung, B. Schornstein, B. Bonakdarpour, and M. Herlihy. Distributed runtime verification of metric temporal properties for cross-chain protocols. In *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022*, pages 23–33. IEEE, 2022.

[50] R. Gay, H. Mantel, and B. Sprick. Service automata. In G. Barthe, A. Datta, and S. Etalle, editors, *Formal Aspects of Security and Trust - 8th International Workshop, FAST 2011, Leuven, Belgium, September 12-14, 2011. Revised Selected Papers*, volume 7140 of *Lecture Notes in Computer Science*, pages 148–163. Springer, 2011.

[51] P. B. Gibbons and E. Korach. Testing shared memories. *SIAM J. Comput.*, 26(4):1208–1244, 1997.

[52] É. Goubault, J. Ledent, and S. Mimram. Concurrent specifications beyond linearizability. In J. Cao, F. Ellen, L. Rodrigues, and B. Ferreira, editors, *22nd International Conference on Principles of Distributed Systems, OPODIS 2018, December 17-19, 2018, Hong Kong, China*, volume 125 of *LIPIcs*, pages 28:1–28:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[53] R. Guerraoui and E. Ruppert. Linearizability is not always a safety property. In G. Noubir and M. Raynal, editors, *Networked Systems - Second International Conference, NETYS 2014, Marrakech, Morocco, May 15-17, 2014. Revised Selected Papers*, volume 8593 of *Lecture Notes in Computer Science*, pages 57–69. Springer, 2014.

[54] A. Haeberlen, P. Kouznetsov, and P. Druschel. Peerreview: practical accountability for distributed systems. In T. C. Bressoud and M. F. Kaashoek, editors, *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*, pages 175–188. ACM, 2007.

[55] S. Hallé, R. Khoury, Q. Betti, A. El-Hokayem, and Y. Falcone. Decentralized enforcement of document lifecycle constraints. *Inf. Syst.*, 74(Part):117–135, 2018.

[56] K. Havelund and A. Goldberg. Verify your runs. In B. Meyer and J. Woodcock, editors, *Verified Software: Theories, Tools, Experiments, First IFIP TC 2/WG 2.3 Conference, VSTTE 2005, Zurich, Switzerland, October 10-13, 2005, Revised Selected Papers and Discussions*, volume 4171 of *Lecture Notes in Computer Science*, pages 374–383. Springer, 2005.

[57] K. Havelund and G. Rosu. An overview of the runtime verification tool java pathexplorer. *Formal Methods Syst. Des.*, 24(2):189–215, 2004.

[58] T. A. Henzinger, C. M. Kirsch, H. Payer, A. Sezgin, and A. Sokolova. Quantitative relaxation of concurrent data structures. In R. Giacobazzi and R. Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, pages 317–328. ACM, 2013.
[59] M. Herlihy. Wait-free synchronization. *ACM Trans. Program. Lang. Syst.*, 13(1):124–149, 1991.
[60] M. Herlihy, D. N. Kozlov, and S. Rajsbaum. *Distributed Computing Through Combinatorial Topology.* Morgan Kaufmann, 2013.
[61] M. Herlihy and N. Shavit. *The art of multiprocessor programming.* Morgan Kaufmann, 2008.
[62] M. Herlihy and J. M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, 1990.
[63] M. Inoue and W. Chen. Linear-time snapshot using multi-writer multi-reader registers. In G. Tel and P. M. B. Vitányi, editors, *Distributed Algorithms, 8th International Workshop, WDAG '94, Terschelling, The Netherlands, September 29 - October 1, 1994, Proceedings*, volume 857 of *Lecture Notes in Computer Science*, pages 130–140. Springer, 1994.
[64] G. V. R. Jiménez. *Verificación de la linealizabilidad en tiempo de ejecución (in Spanish).* Master Thesis. Posgrado en Ciencia e Ingeniería de la Computación. UNAM, 2022.
[65] S. Kazemlou and B. Bonakdarpour. Crash-resilient decentralized synchronous runtime verification. In *37th IEEE Symposium on Reliable Distributed Systems, SRDS 2018, Salvador, Brazil, October 2-5, 2018*, pages 207–212. IEEE Computer Society, 2018.
[66] D. Kini, U. Mathur, and M. Viswanathan. Dynamic race prediction in linear time. In A. Cohen and M. T. Vechev, editors, *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, pages 157–170. ACM, 2017.
[67] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, 1978.
[68] L. Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Trans. Computers*, 28(9):690–691, 1979.
[69] L. Lamport. On interprocess communication. part I: basic formalism. *Distributed Comput.*, 1(2):77–85, 1986.
[70] L. Lamport. On interprocess communication. part II: algorithms. *Distributed Comput.*, 1(2):86–101, 1986.
[71] M. Leucker and C. Schallhart. A brief account of runtime verification. *J. Log. Algebraic Methods Program.*, 78:293–303, 2009.
[72] J. M. Lourenço, J. Fiedor, B. Krena, and T. Vojnar. Discovering concurrency errors. In E. Bartocci and Y. Falcone, editors, *Lectures on Runtime Verification - Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 34–60. Springer, 2018.
[73] Q. Luo and G. Rosu. Enforcemop: a runtime property enforcement system for multithreaded programs. In M. Pezzè and M. Harman, editors, *International Symposium on Software Testing and Analysis, ISSTA '13, Lugano, Switzerland, July 15-20, 2013*, pages 156–166. ACM, 2013.
[74] N. A. Lynch and F. W. Vaandrager. Forward and backward simulations: I. untimed systems. *Inf. Comput.*, 121(2):214–233, 1995.
[75] U. Mathur, D. Kini, and M. Viswanathan. What happens-after the first race? enhancing the predictive power of happens-before based dynamic race detection. *Proc. ACM Program. Lang.*, 2(OOPSLA):145:1–145:29, 2018.
[76] U. Mathur, A. Pavlogiannis, and M. Viswanathan. Optimal prediction of synchronization-preserving races. *Proc. ACM Program. Lang.*, 5(POPL):1–29, 2021.
[77] P. O. Meredith, D. Jin, D. Griffith, F. Chen, and G. Rosu. An overview of the MOP runtime verification framework. *Int. J. Softw. Tools Technol. Transf.*, 14(3):249–289, 2012.
[78] M. Moir and N. Shavit. Concurrent data structures. In D. P. Mehta and S. Sahni, editors, *Handbook of Data Structures and Applications.* Chapman and Hall/CRC, 2004.
[79] M. Mostafa and B. Bonakdarpour. Decentralized runtime verification of LTL specifications in distributed systems. In *2015 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2015, Hyderabad, India, May 25-29, 2015*, pages 494–503. IEEE Computer Society, 2015.
[80] H. Nazarpour, Y. Falcone, S. Bensalem, and M. Bozga. Concurrency-preserving and sound monitoring of multi-threaded component-based systems: theory, algorithms, implementation, and evaluation. *Formal Aspects Comput.*, 29(6):951–986, 2017.
[81] G. Neiger. Set-linearizability. In J. H. Anderson, D. Peleg, and E. Borowsky, editors, *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing, Los Angeles, California, USA, August 14-17, 1994*, page 396. ACM, 1994.
[82] C. H. Papadimitriou. The serializability of concurrent database updates. *J. ACM*, 26(4):631–653, 1979.
[83] A. Pavlogiannis. Fast, sound, and effectively complete dynamic race prediction. *Proc. ACM Program. Lang.*, 4(POPL):17:1–17:29, 2020.
[84] S. Qadeer and S. Tasiran. Runtime verification of concurrency-specific correctness criteria. *Int. J. Softw. Tools Technol. Transf.*, 14(3):291–305, 2012.

[85] M. Raynal. *Concurrent Programming - Algorithms, Principles, and Foundations*. Springer, 2013.
[86] A. Rinberg and I. Keidar. Intermediate value linearizability: A quantitative correctness criterion. In H. Attiya, editor, *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, volume 179 of *LIPIcs*, pages 2:1–2:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
[87] G. V. Rodríguez and A. Castañeda. Towards efficient runtime verified linearizable algorithms. In E. Ábrahám and H. Abbas, editors, *Runtime Verification - 24th International Conference, RV 2024, Istanbul, Turkey, October 15-17, 2024, Proceedings*, volume 15191 of *Lecture Notes in Computer Science*, pages 262–281. Springer, 2024.
[88] V. Roussanaly and Y. Falcone. Decentralised runtime verification of timed regular expressions. In A. Artikis, R. Posenato, and S. Tonetta, editors, *29th International Symposium on Temporal Representation and Reasoning, TIME 2022, November 7-9, 2022, Virtual Conference*, volume 247 of *LIPIcs*, pages 6:1–6:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
[89] M. Samadi, F. Ghassemi, and R. Khosravi. Decentralized runtime enforcement of message sequences in message-based systems. In Q. Bramas, R. Oshman, and P. Romano, editors, *24th International Conference on Principles of Distributed Systems, OPODIS 2020, December 14-16, 2020, Strasbourg, France (Virtual Conference)*, volume 184 of *LIPIcs*, pages 21:1–21:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
[90] C. Sánchez, G. Schneider, W. Ahrendt, E. Bartocci, D. Bianculli, C. Colombo, Y. Falcone, A. Francalanza, S. Krstic, J. M. Lourenço, D. Nickovic, G. J. Pace, J. Rufino, J. Signoles, D. Traytel, and A. Weiss. A survey of challenges for runtime verification from advanced application domains (beyond software). *Formal Methods Syst. Des.*, 54(3):279–335, 2019.
[91] A. D. Sarma, S. Holzer, L. Kor, A. Korman, D. Nanongkai, G. Pandurangan, D. Peleg, and R. Wattenhofer. Distributed verification and hardness of distributed approximation. *SIAM J. Comput.*, 41(5):1235–1265, 2012.
[92] S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. E. Anderson. Eraser: A dynamic data race detector for multithreaded programs. *ACM Trans. Comput. Syst.*, 15(4):391–411, 1997.
[93] T. Scheffel and M. Schmitz. Three-valued asynchronous distributed runtime verification. In *Twelfth ACM/IEEE International Conference on Formal Methods and Models for Codesign, MEMOCODE 2014, Lausanne, Switzerland, October 19-21, 2014*, pages 52–61. IEEE, 2014.
[94] F. B. Schneider. Enforceable security policies. *ACM Trans. Inf. Syst. Secur.*, 3(1):30–50, 2000.
[95] G. Sela, M. Herlihy, and E. Petrank. Brief announcement: Linearizability: A typo. In A. Miller, K. Censor-Hillel, and J. H. Korhonen, editors, *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, pages 561–564. ACM, 2021.
[96] K. Sen, A. Vardhan, G. Agha, and G. Rosu. Efficient decentralized monitoring of safety in distributed systems. In A. Finkelstein, J. Estublier, and D. S. Rosenblum, editors, *26th International Conference on Software Engineering (ICSE 2004), 23-28 May 2004, Edinburgh, United Kingdom*, pages 418–427. IEEE Computer Society, 2004.
[97] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath. BFT protocol forensics. In Y. Kim, J. Kim, G. Vigna, and E. Shi, editors, *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 1722–1743. ACM, 2021.
[98] C. Soueidi, A. El-Hokayem, and Y. Falcone. Opportunistic monitoring of multithreaded programs. In L. Lambers and S. Uchitel, editors, *Fundamental Approaches to Software Engineering - 26th International Conference, FASE 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings*, volume 13991 of *Lecture Notes in Computer Science*, pages 173–194. Springer, 2023.
[99] Y. Wei, N. Ben-David, G. E. Blelloch, P. Fatourou, E. Ruppert, and Y. Sun. Constant-time snapshots with applications to concurrent data structures. In J. Lee and E. Petrank, editors, *PPoPP '21: 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Virtual Event, Republic of Korea, February 27- March 3, 2021*, pages 31–46. ACM, 2021.